



MODUL PRAKTIKUM MATA KULIAH

Analitika Perkotaan

Semester Ganjil Tahun Akademik 2026/2027

Prof. Ibnu Syabri, B.Sc., M.Sc., Ph.D
Muhammad Abdul Mubdi Bindar, S.T., M.T.
Ahmad Luky Ramdani, S.Komp., M.Kom.
Valendya Rilansari, S.P.W.K., M.P.W.K.
Chania Rahmah, S.P.W.K., M.Sc.

Program Studi Perencanaan Wilayah dan Kota
Fakultas Teknologi Infrastruktur dan Kewilayahan
Institut Teknologi Sumatera
2026

	INSTITUT TEKNOLOGI SUMATERA	Nomor :
		Tanggal:
	PENGESEHAN MODUL PRAKTIKUM ANALITIKA PERKOTAAN	Revisi :
		Halaman:

MODUL PRAKTIKUM MATA KULIAH ANALITIKA PERKOTAAN
 PROGRAM STUDI PERENCANAAN WILAYAH DAN KOTA
 FAKULTAS TEKNOLOGI INFRASTRUKTUR DAN KEWILAYAHAN
 INSTITUT TEKNOLOGI SUMATERA

PROSES	PENANGGUNG JAWAB			TANGGAL
	Nama	Jabatan	Tanda Tangan	
Disusun oleh	Muhammad Abdul Mubdi Bindar, S.T., M.T.	Dosen Penanggung Jawab		13 Juli 2026
Diperiksa oleh	Chania Rahmah, S.P.W.K., M.Sc.	Gugus Kendali Mutu Prodi		20 Juli 2026
Pengesahan	Dr. nat. techn. Shahnaz Nabila Fuady, S.T., M.T.	Koordinator Program Studi Perencanaan Wilayah dan Kota		27 Juli 2026

Daftar Isi

Kata Pengantar	1
Cara Memakai Modul Ini	1
Dataset	1
Perangkat Lunak	1
1 Fondasi Lingkungan Kerja: Python <i>Offline</i> , Terminal, dan Direktori Proyek	2
1.1 Pendahuluan	2
1.2 Gambaran Umum Kasus	2
1.3 Instruksi	2
1.4 Latihan Mandiri	17
2 Pengolahan Data Terstruktur dengan Pandas	19
2.1 Pendahuluan	19
2.2 Gambaran Umum Kasus	19
2.3 Instruksi	19
2.4 Latihan Mandiri	29
3 Pengolahan Data Terstruktur dengan SQL (SQLite)	30
3.1 Pendahuluan	30
3.2 Gambaran Umum Kasus	30
3.3 Instruksi	30
3.4 Latihan Mandiri	41
4 <i>Exploratory Data Analysis</i> (EDA)	42
4.1 Pendahuluan	42
4.2 Gambaran Umum Kasus	42
4.3 Instruksi	42
4.4 Latihan Mandiri	52
5 Visualisasi Data dengan Matplotlib dan Seaborn	54
5.1 Pendahuluan	54
5.2 Gambaran Umum Kasus	54
5.3 Instruksi	54
5.4 Latihan Mandiri	63

Kata Pengantar

<masukkan kata pengantar dari tim dosen di sini>

Modul praktikum ini menemani perkuliahan **Analitika Perkotaan (PL25-41003)** di Program Studi Perencanaan Wilayah dan Kota, Institut Teknologi Sumatera. Setiap modul dirancang untuk satu pertemuan praktikum (150 menit) dan mengikuti satu kasus besar yang sama: **Kota Bandar Lampung**.

Cara Memakai Modul Ini

Setiap modul memiliki susunan yang sama sehingga ritme belajar Anda terjaga dari pekan ke pekan:

1. **Sub-CPMK** — kemampuan yang menjadi target pertemuan tersebut.
2. **Pendahuluan** — mengapa kemampuan ini penting bagi seorang perencana.
3. **Gambaran Umum Kasus** — cerita kasus perkotaan yang akan kita kerjakan.
4. **Instruksi** — langkah-langkah praktik yang Anda ikuti dan ketik sendiri di kelas.
5. **Latihan Mandiri** — tugas dengan alur yang sama persis seperti Instruksi, tetapi memakai kolom data yang berbeda. Inilah cara Anda menguji pemahaman tanpa bimbingan.

Sepanjang modul, Anda akan menemui kotak **callout** seperti ini:

Konsep Python: Contoh Callout

Kotak seperti ini muncul **tepat setelah** sebuah konsep Python dipakai untuk pertama kalinya di dalam instruksi. Isinya penjelasan konsep tersebut, untuk Anda baca ulang secara mandiri di luar kelas. Saat praktikum, Anda tidak perlu berhenti lama di sini.

Dataset

Seluruh praktikum memakai dataset yang dibagikan dosen dalam folder [dataset-ganjil-2026-2027](#). Sumber utamanya adalah data **PODES 2021** (BPS) untuk Kota Bandar Lampung dan Kota Metro, serta data satuan pendidikan dari Kemendikdasmen. Kami berterima kasih kepada **Prof. Ibnu Syabri, B.Sc., M.Sc., Ph.D.** atas penyediaan dataset PODES 2021 untuk pembelajaran praktikum ini. Cara menata folder dataset ini di komputer Anda dijelaskan pada Modul 1.

Perangkat Lunak

Praktikum memakai perangkat lunak yang seluruhnya gratis: **Miniconda** (pengelola lingkungan Python), **Visual Studio Code** (penyunting kode), dan sejumlah *library* Python yang akan dipasang bertahap sesuai kebutuhan tiap modul. Semua langkah penyiapannya dituntun pada Modul 1.

1 Fondasi Lingkungan Kerja: Python *Offline*, Terminal, dan Direktori Proyek

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu menyiapkan lingkungan kerja Python *offline* (Conda), mengoperasikan perintah terminal dasar, mengelola berkas dengan struktur direktori proyek standar studio, serta membuat dan menyambungkan *kernel* Jupyter Notebook.

1.1 Pendahuluan

<masukkan narasi yang menceritakan ‘big why’ menyiapkan fondasi lingkungan kerja analitika perkotaan di sini>

1.2 Gambaran Umum Kasus

Setiap kali sebuah studio perencanaan memulai proyek baru, hal pertama yang dikerjakan bukanlah menggambar. Meja gambar dirapikan, map-map berkas disiapkan dan diberi label, lalu penggaris, pensil, dan kalkulator ditata di rak masing-masing. Tanpa persiapan itu, di tengah proyek kita akan kehilangan berkas, meminjam alat orang lain yang ternyata berbeda ukuran, dan hasil kerja menjadi kacau.

Pekerjaan analitika perkotaan persis sama. Sebelum menyentuh data Kota Bandar Lampung pada pertemuan-pertemuan berikutnya, dalam modul ini kita merapikan “meja kerja digital” terlebih dahulu. Kita akan menata direktori proyek dengan struktur yang baku, memasang peralatan Python secara *offline* menggunakan Conda agar tidak bergantung pada koneksi internet yang tidak stabil, belajar berbicara dengan komputer lewat terminal, menulis skrip Python pertama bernama `sapa.py`, berlatih membaca pesan kegagalan (*error*) yang paling sering muncul, membuktikan mengapa satu berkas skrip harus lengkap berdiri sendiri, dan menutup pertemuan dengan menyiapkan “buku catatan lapangan” digital kita: Jupyter Notebook yang tersambung ke mesin Python (*kernel*) milik kita sendiri.

1.3 Instruksi

1.3.a Menata Meja Kerja: Membuat Struktur Direktori Proyek Studio

Bayangkan hari pertama masuk studio: kita diberi satu laci khusus, lalu di dalamnya kita menyiapkan map-map bertuliskan “data mentah”, “gambar kerja”, dan “laporan”. Di komputer, laci itu bernama **direktori** (sebutan lain untuk *folder*), dan seluruh pekerjaan satu semester ini akan tinggal di dalam satu direktori induk.

Ikuti langkah berikut:

1. Buka **File Explorer**, lalu masuk ke folder `C: → Users → <Nama User Anda> → Documents`.

⚠️ Hindari Folder Downloads

Jangan gunakan folder seperti `Downloads` sebagai direktori proyek utama. Berkas di folder tersebut rentan terhapus secara tidak sengaja ketika Anda melakukan pembersihan berkas sementara atau pemeliharaan sistem komputer.

i Pengetahuan Komputer

Root Directory dan Media Penyimpanan (HDD/SSD)

Pangkal teratas dari struktur folder di komputer disebut **direktori akar** (*root directory*). Di sistem operasi Windows, direktori akar ini direpresentasikan oleh huruf kandar (*drive letter*) seperti `C:`, `D:`, atau `E:`.

Kandar `C:` umumnya merupakan partisi utama pada media penyimpanan permanen tempat sistem operasi Windows terpasang. Komputer modern saat ini umumnya menggunakan **SSD (Solid-State Drive)** yang memiliki kecepatan baca-tulis jauh lebih cepat dibandingkan **HDD (Hard Disk Drive)** mekanis lama. Memahami struktur ini sangat penting agar kita tahu persis di mana data fisik kita disimpan di dalam perangkat keras komputer.

2. Buat folder baru bernama `analitika-perkotaan`. Inilah **direktori proyek** kita, padanan laci studio tadi.
3. Di dalamnya, buat empat subfolder sehingga susunannya seperti ini:

```
analitika-perkotaan/      <- direktori proyek (laci induk)
├─ dataset-ganjil-2026-2027/ <- map data mentah dari dosen
├─ modul/                 <- map buku catatan (notebook) praktikum
├─ skrip/                  <- map skrip Python (.py)
└─ keluaran/               <- map hasil kerja (tabel, grafik, peta)
```

4. Salin folder `dataset-ganjil-2026-2027` yang dibagikan dosen ke dalam direktori proyek, sehingga namanya persis seperti bagan di atas.

i Pengetahuan Komputer

Padanan Path dan Struktur Folder Tree

Struktur folder tree di atas dapat direpresentasikan ke dalam bentuk **path**. Sebagai contoh, struktur folder tree berikut:

```
C:
├─ Users
│   └─ Aulia
│       └─ Documents
│           └─ analitika-perkotaan
```

akan dituliskan sebagai path absolut: `C:/Users/Aulia/Documents/analitika-perkotaan`.

Mengapa pemahaman path ini penting?

Perhatikan contoh penamaan direktori dengan path yang salah di bawah ini:

```
C:
├── Users
│   ├── Aulia
│   │   ├── Documents
│   │   │   └── analitika-perkotaan
│   │   ├── Music
│   │   │   └── analitika-perkotaan
```

Meskipun kedua folder sama-sama bernama `analitika-perkotaan`, path absolutnya sangat berbeda: yang satu berada di `C:/Users/Aulia/Documents/analitika-perkotaan` dan yang satu lagi di `C:/Users/Aulia/Music/analitika-perkotaan`. Jika program kita salah mencari lokasi, ia tidak akan menemukan berkas yang tepat.

i Pengetahuan Komputer

Penggunaan Simbol Slash di Windows vs Unix untuk penulisan *Path*

Sistem operasi Windows secara tradisional menggunakan simbol *backslash* (`\`) untuk memisahkan nama folder di dalam path (misalnya `C:\Users\Aulia`). Namun, sistem berbasis Unix (seperti Linux dan macOS), serta browser web, menggunakan *forward slash* (`/`) (misalnya `/Users/Aulia`). Dalam pemrograman Python, disarankan untuk menggunakan *forward slash* (`/`) karena Python dapat menerjemahkannya secara otomatis di Windows dan mencegah galat akibat karakter khusus (*escape character*).

Susunan bercabang seperti bagan di atas disebut **folder tree** (pohon direktori). Untuk menunjuk lokasi sebuah berkas di dalam pohon itu, komputer memakai **path**: rangkaian nama folder yang dilalui untuk sampai ke berkas tersebut, dipisahkan garis miring. Ada dua cara menuliskannya:

- **Path lengkap (absolut)** — dari pangkal pohon sampai ujung, misalnya `C:/Users/Aulia/Documents/analitika-perkotaan/skrip/sapa.py`. Seperti alamat surat lengkap: kota, kecamatan, jalan, nomor rumah.
- **Path relatif** — dihitung dari posisi kita berdiri sekarang. Jika kita sedang berada di dalam `analitika-perkotaan`, berkas yang sama cukup ditunjuk dengan `skrip/sapa.py`. Seperti berkata “dua rumah di sebelah kanan” kepada orang yang sudah berdiri di jalan yang sama.

Sepanjang semester kita akan lebih sering memakai *path* relatif, dan karena itu **posisi kita berdiri** (folder mana yang sedang terbuka) menjadi penting. Ini akan segera terasa saat kita bekerja dengan terminal.

1.3.b Menyiapkan Ruang Kerja: *Environment* Mandiri dan Instalasi Conda

Di studio, setiap perencana punya ruang kerja studio pribadi berisi meja, kursi, dan rak peralatan miliknya sendiri, supaya pengaturan alat untuk proyek yang satu tidak tercampur dengan proyek lain. Dalam dunia Python, ruang kerja pribadi itu bernama **environment**, dan petugas yang menyiapkannya bernama **Conda**. Satu komputer bisa memiliki banyak *environment* sekaligus, masing-masing berisi versi Python dan *library* yang berbeda, tanpa saling mengganggu.

Langkah nol: periksa dulu penghuni lama ruangan. Sebelum menata meja kerja, peserta studio memeriksa dulu meja siapa saja yang sudah menumpuk di ruangan itu — supaya tidak keliru memakai meja orang lain, dan tidak menggeser barang yang ternyata masih dipakai. Di komputer, langkah itu berarti mendata Python apa saja yang **sudah ada** sebelum kita menambah apa pun. Buka **Command Prompt** atau **PowerShell**, lalu ketik dua perintah berikut satu per satu. Tuliskan persis seperti tertulis, lengkap dengan akhiran `.exe`-nya:

```
where.exe python
where.exe conda
```

Perintah `where.exe` menampilkan **semua** lokasi program bernama itu yang ditemukan komputer, berurutan dari yang paling didahulukan. Salin hasilnya ke buku catatan Anda — kita akan menagihnya lagi di langkah kelima. Jangan kaget bila keluarannya lebih dari satu baris; justru itu yang biasa terjadi.

⚠ Penting!

Mengapa Ditulis `where.exe`, Bukan `where`

Windows menyediakan dua terminal yang tampilannya nyaris serupa: **Command Prompt** dan **PowerShell**. Bedakan dari tulisan di awal barisnya — PowerShell selalu mengawali barisnya dengan `PS`, seperti `PS C:\Users\Nama>`, sedangkan Command Prompt langsung `C:\Users\Nama>`. Perhatikan ini baik-baik, karena pada Windows 11 PowerShell-lah yang paling sering terbuka lebih dulu — termasuk saat kita membuka terminal bawaan VS Code nanti.

Perbedaannya berakibat langsung di langkah ini. Di Command Prompt, `where` adalah nama program pencari lokasi yang kita maksud. Di PowerShell, nama `where` sudah dipakai lebih dulu sebagai singkatan perintah bawaannya sendiri (`Where-Object`) yang tugasnya menyaring daftar — sama sekali tidak berhubungan dengan mencari lokasi program.

Bentuk kegagalannya justru yang paling menyesatkan: PowerShell **tidak menampilkan pesan galat apa pun**. Ia hanya diam, kursor kembali ke baris kosong, seolah-olah perintahnya sudah dikerjakan. Layar kosong itu sangat mudah dibaca sebagai baris terakhir tabel di bawah — “belum ada Python” — padahal komputer Anda bisa jadi punya tiga sekaligus. Berbeda dengan pesan `'...' is not recognized` yang jelas-jelas menyuruh kita memperbaiki sesuatu, kegagalan yang diam justru membuat kita melangkah ke tahap berikutnya dengan kesimpulan yang keliru.

Akhiran `.exe` menutup celah itu: ia memaksa terminal menjalankan program pencariannya, bukan singkatan tadi. Bentuk `where.exe` **aman dipakai di ketiganya** — Command Prompt, PowerShell, maupun Anaconda Prompt — jadi biasakan menulis akhiran itu sepanjang mata kuliah ini.

Sekarang cocokkan keluaran Anda dengan tabel berikut, supaya langkah nol ini menghasilkan sebuah **kesimpulan**, bukan sekadar tumpukan tulisan:

Yang muncul di keluaran	Artinya
<code>C:\Python27\ArcGIS10.8\...</code> atau <code>... \ArcGISx6410.8\...</code>	ArcGIS Desktop 10.x — membawa Python 2.7
<code>...\ArcGIS\Pro\bin\Python\...</code> (<code>where.exe conda</code>)	(pada ArcGIS Pro — membawa Conda sendiri
<code>...\WindowsApps\python.exe</code>	Python dari Microsoft Store

Yang muncul di keluaran	Artinya
C:\Python3xx\... atau ... \\AppData\\Local\\Programs\\Python\\...	Python dari situs python.org
tidak ada keluaran sama sekali	belum ada Python — kondisi paling bersih; pastikan dulu yang Anda ketik <code>where.exe</code> , bukan <code>where</code>

⚠ Komputer Anda Mungkin Sudah Punya Python Tanpa Anda Sadari

Bila di komputer Anda terpasang **ArcGIS**, maka Python **sudah ada di sana sejak lama** — dan ada dua kemungkinan yang penanganannya berbeda. Tabel di atas menunjukkan Anda termasuk yang mana. Baca **hanya kasus Anda**; bila keduanya muncul, kerjakan keduanya.

Kasus A — ArcGIS Desktop 10.x (ArcMap). Yang dibawahnya adalah Python **versi 2.7**, umumnya di `C:/Python27/ArcGIS10.8`, dan ia sama sekali tidak memakai Conda. Apakah Python 2.7 ini terdaftar lebih dulu di daftar alamat komputer **tidak bisa ditebak dari luar**: pada sebagian mesin ia menang, pada sebagian lain ia tidak terdaftar sama sekali meski ArcGIS-nya terpasang. Bila ia menang, mengetik `python` di terminal baru menjalankan **Python 2.7** — bukan Python kita. Bahayanya justru bukan karena ia menolak bekerja, melainkan sebaliknya: ia mau berjalan, lalu gagal di tempat yang membingungkan. Kode seperti `print(f"Halo, {nama}")` yang akan kita tulis sebentar lagi ditolaknya dengan `SyntaxError`, padahal kode itu benar. Anda akan mengira diri sendiri salah ketik, padahal yang keliru adalah Python yang menjalankannya. Inilah alasan langkah nol dikerjakan sebelum apa pun.

Kasus B — ArcGIS Pro. ArcGIS Pro membawa **Conda-nya sendiri**, lengkap dengan sebuah *environment* bernama `arcgispro-py3` di `C:/Program Files/ArcGIS/Pro/bin/Python/envs/arcgispro-py3`. Bahayanya lebih halus dan baru terasa di langkah pertama nanti: perintah `conda` yang dikenali komputer Anda bisa jadi **milik ArcGIS Pro**, sehingga ruang kerja mata kuliah ini tanpa sadar dibangun di dalam instalasi ArcGIS. Periksa keluaran `where.exe conda` Anda: bila ada baris yang memuat potongan `\\ArcGIS\\Pro\\`, jangan pakai terminal “Python Command Prompt” bawaan ArcGIS — gunakan **Anaconda Prompt** milik Miniconda. Bila Anaconda Prompt belum ada di komputer Anda, pasang Miniconda dari berkas yang dibagikan dosen terlebih dahulu.

Berlaku untuk kedua kasus: jangan dihapus, dan jangan dipakai untuk mata kuliah ini. Jangan dihapus, karena ArcGIS bergantung sepenuhnya pada Python bawaannya — merusaknya berarti merusak ArcGIS Anda. Jangan dipakai, dan ini bukan soal selera: Python 2.7 pada Kasus A sudah berhenti dikembangkan sejak 1 Januari 2020 sehingga tidak satu pun *library* yang kita pakai semester ini berjalan di sana, sedangkan *environment* bawaan ArcGIS Pro pada Kasus B memang dirancang untuk tidak diubah-ubah. *Environment* yang kita buat hari ini berdiri **berdampingan** dengan Python-Python itu: menambah satu ruang kerja baru, tanpa menggusur ruang kerja yang sudah ada.

Di luar dua kasus itu, Python juga bisa masuk lewat **Microsoft Store** atau pemasangan dari situs **python.org**, sering kali tanpa kita sadari. Jangan buru-buru menganggapnya sepele: pada komputer yang sudah diperiksa untuk mata kuliah ini, justru Python dari python.org inilah yang **memenangkan baris pertama** daftar alamat — mengalahkan Python bawaan ArcGIS sekalipun. Bahayanya berkebalikan dengan Kasus A, dan justru karena itu lebih licin: ia **tidak menolak apa pun**. Kode Anda jalan, tidak ada tulisan merah, tetapi *library* yang dipakainya berbeda versi sehingga angka yang keluar bisa berbeda dari yang muncul di layar teman sebelah — tanpa satu pun pesan

yang memberi tahu. Kegagalan yang diam jauh lebih sulit dilacak daripada yang berisik. Karena itu keberadaan mereka tidak cukup diingat, melainkan harus dibuktikan siapa yang sesungguhnya berjalan — dan itulah yang kita kerjakan di langkah kelima nanti.

Langkah pertama: periksa dulu, jangan langsung memasang. Komputer laboratorium kampus sudah dipasang Conda (bahkan lengkap dengan *environment* mata kuliah ini) oleh tim dosen, jadi memasang ulang hanya membuang waktu. Buka **Anaconda Prompt**.

⚠ Penting!

Konsep Path & Menjalankan Conda di Terminal Biasa

Anaconda Prompt adalah terminal khusus yang sudah dikonfigurasi secara otomatis agar dapat mengenali perintah `conda`. Jika Anda ingin menjalankan perintah `conda` dari Command Prompt biasa atau PowerShell, Anda harus menginisialisasinya terlebih dahulu dengan menjalankan perintah `conda init` di Anaconda Prompt, lalu menutup dan membuka kembali terminal biasa Anda.

Setelah memahami hal ini, cobalah buka Command Prompt biasa atau PowerShell di komputer Anda, lalu ketik `conda --version` untuk memverifikasi apakah ia sudah mengenali perintah tersebut.

Di Anaconda Prompt, ketik:

```
conda --version
```

Ada dua kemungkinan jawaban:

- Muncul tulisan seperti `conda 24.11.0` (angkanya boleh berbeda) — Conda **sudah terpasang**. Lanjutkan ke pemeriksaan *environment* di bawah.
- Muncul pesan seperti `'conda' is not recognized ...` — Conda **belum ada**. Pasang **Miniconda** dari berkas instalasi yang dibagikan dosen (tersedia *offline*, tidak perlu mengunduh), ikuti pilihan bawaannya sampai selesai, tutup terminalnya, buka **Anaconda Prompt** yang baru, lalu ulangi perintah di atas sampai versi Conda muncul.

Langkah kedua: periksa apakah *environment* mata kuliah sudah ada. Ketik:

```
conda env list
```

Perintah ini menampilkan daftar semua ruang kerja (*environment*) yang ada di komputer. Jika di daftarnya sudah ada baris bernama `analitika-perkotaan` (kondisi normal di komputer lab), ruang kerja kita sudah berdiri — lompat ke langkah keempat.

Langkah ketiga (hanya jika belum ada): buat *environment* dari berkas resep. Dosen membagikan sebuah berkas bernama `environment.yml`. Salin berkas itu ke direktori proyek `analitika-perkotaan` yang kita buat tadi. Isinya seperti ini:

```
name: analitika-perkotaan

channels:
  - conda-forge
```

```

- nodefaults

dependencies:
# Dasar: Modul 2-4
- python=3.12
- pandas
- openpyxl

# Visual: Modul 5
- matplotlib-base
- seaborn

# Spasial: Modul 6-7
- geopandas
- leafmap
- mapclassify
- folium
- contextily

# Penarikan data: Modul 9
- requests
- beautifulsoup4
- osmnx

# Pemodelan: Modul 10
- scikit-learn

# Dashboard: Tugas Besar Pekan 11-16 & UAS
- streamlit
- streamlit-folium

# Infrastruktur agar notebook berjalan
- ipykernel
- jupyterlab
- notebook
- nbformat
- nbclient
- pyyaml

# Bantu: pembacaan dataset dan tabel interaktif
- dbfread
- itables

variables:
PYTHONNOUSERSITE: "1"

```

Berkas ini adalah **daftar belanja perabot** untuk membangun ruang kerja satu semester: nama ruang kerjanya (**name**), toko tempat membelinya (**channels**), daftar rak, kursi, serta meja yang dibeli (**dependencies**), dan satu aturan rumah yang berlaku di dalam ruangan itu (**variables**). Baris

berawalan tanda `#` adalah **komentar**: keterangan untuk manusia, diabaikan komputer. Aturan rumah `PYTHONNOUSERSITE` itu melarang ruang kerja kita mengambil perabot dari sebuah rak titipan bersama di luar ruangan — alasannya dibahas pada *callout* terakhir bagian ini. Dengan satu berkas ini kita tidak perlu mengetik nama *library* satu per satu — cukup satu perintah, dan Conda membelanjakan semuanya:

```
cd Documents\analitika-perkotaan
conda env create -f environment.yml
```

Perintah `cd` (*change directory*) memindahkan posisi berdiri kita ke direktori proyek — ingat pembahasan *path* tadi: `environment.yml` ditunjuk dengan *path* relatif, sehingga kita harus berdiri di folder tempat berkas itu berada. Proses ini butuh beberapa menit.

Langkah keempat: masuk ke ruang kerja. Aktifkan *environment* dengan:

```
conda activate analitika-perkotaan
```

Perhatikan awalan tulisan di terminal berubah dari `(base)` menjadi `(analitika-perkotaan)`. Awalan itu adalah papan nama ruangan: ia memberi tahu ruang kerja mana yang sedang kita buka pintunya untuk mulai bekerja. Mulai sekarang, setiap kali membuka terminal untuk mata kuliah ini, biasakan memeriksa papan nama tersebut.

Langkah kelima: buktikan ruang kerjanya benar-benar terbuka. Papan nama `(analitika-perkotaan)` itu **klaim**, bukan bukti. Papan nama bisa menyala sementara yang benar-benar bekerja adalah Python lain — dan ini bukan kemungkinan teoretis, melainkan kejadian yang wajar di komputer yang sudah punya banyak Python (ingat hasil langkah nol Anda). Karena itu, sekali di awal semester, kita buktikan. Dengan papan nama `(analitika-perkotaan)` menyala, jalankan tiga perintah berikut — perhatikan lagi akhiran `.exe` pada perintah pertama:

```
where.exe python
python -c "import sys; print(sys.prefix)"
python -c "import sys; print(sys.path)"
```

Ruang kerja Anda dinyatakan **lulus** bila ketiga syarat ini terpenuhi:

1. Baris **paling atas** keluaran `where.exe python` memuat potongan `envs\analitika-perkotaan`.
2. Keluaran perintah kedua berakhir dengan `envs\analitika-perkotaan`.
3. Keluaran perintah ketiga **tidak memuat** potongan `AppData\Roaming\Python` di mana pun.

Catat ketiga keluaran itu — Latihan Mandiri butir 5 akan menagihnya.

Jika Ruang Kerja Anda Tidak Lulus

Perintah pertama tidak mengeluarkan apa pun: hampir pasti yang terketik `where`, bukan `where.exe`, dan terminal Anda PowerShell. Ulangi dengan akhiran `.exe`-nya.

Syarat 1 atau 2 gagal (yang muncul Python lain): tutup terminal, buka **Anaconda Prompt** baru, jalankan `conda activate analitika-perkotaan`, lalu ulangi. Bila masih gagal, kemungkinan besar *environment*-nya belum benar-benar terbentuk — ulangi langkah kedua dan ketiga. Bila yang muncul di baris teratas justru `C:\Python27\...`, itu Kasus A pada *callout* sebelumnya — penyebabnya hampir selalu `conda activate` yang belum dijalankan di terminal tersebut.

Syarat 3 gagal (muncul `AppData\Roaming\Python`): **jangan diperbaiki sendiri, laporkan ke dosen.** Kondisi ini berarti ada rak paket titipan yang membayangi ruang kerja Anda, dan membersihkannya sembarangan bisa merusak program lain di komputer Anda — termasuk ArcGIS Pro bila Anda memakainya.

Pengetahuan Komputer

Satu Komputer, Banyak Python: Mengapa Harus Dibuktikan

Ketika kita mengetik `python` di terminal, komputer **tidak tahu** Python mana yang kita maksud. Ia hanya menyusuri sebuah daftar alamat bernama **PATH** dari atas ke bawah, lalu menjalankan **program pertama** bernama `python` yang ia temukan. Itulah sebabnya `where.exe python` bisa menampilkan beberapa baris sekaligus: baris pertama adalah yang menang, sisanya menganggur.

`conda activate` bekerja dengan cara menyelipkan alamat *environment* kita ke **urutan paling atas** daftar itu. Bila ada sesuatu yang menyelipkan alamat lain lebih atas lagi, papan nama tetap menyala tetapi yang berjalan Python yang lain. Analogi studionya: papan nama di pintu ruangan bisa saja tertulis nama Anda, sementara yang duduk mengerjakan gambar di dalamnya orang lain. Satu-satunya cara memastikan adalah menengok ke dalam — dan `where.exe python` beserta `sys.prefix` adalah cara menengoknya.

Perintah `python -c "..."` sendiri berarti “jalankan kode di dalam tanda kutip ini, sekali saja, lalu keluar”. Isi kodenya akan Anda pahami sepenuhnya di Modul 2; untuk sekarang ia cukup diperlakukan sebagai **alat periksa**, seperti meteran yang kita pakai tanpa perlu tahu isi mesinnya.

Jangan Pernah Memasang Paket di Luar *Environment*

Suatu hari Anda akan menemukan tutorial di internet yang menyuruh mengetik `pip install nama-paket`. **Jangan pernah menjalankannya sebelum papan nama (*analitika-perkotaan*) menyala.**

Alasannya begini. Selain rak di dalam tiap ruang kerja, Windows menyediakan **satu rak titipan bersama** di `C:/Users/<Nama User Anda>/AppData/Roaming/Python/`. Rak itu dipakai bersama-sama oleh **semua** Python bernomor versi sama di komputer itu — termasuk Python bawaan program lain seperti ArcGIS Pro, bila nomor versinya kebetulan sama. Lebih berbahaya lagi: isi rak titipan itu **didahulukan** atas isi ruang kerja kita.

Akibatnya, satu perintah `pip install` yang dijalankan tanpa mengaktifkan *environment* bisa: (a) diam-diam mengubah perilaku ruang kerja mata kuliah ini, (b) mengganggu program lain yang memakai Python bernomor versi sama, dan (c) melakukan keduanya **tanpa satu pun pesan galat** — sehingga kerusakannya baru terasa berminggu-minggu kemudian dan sangat sulit dilacak.

Aturan kelas ini sederhana dan tidak ada pengecualiannya: `conda activate analitika-perkotaan` dulu, baru memasang apa pun.

1.3.c Berbicara dengan Terminal: Menjalankan Perintah Python Interaktif

Terminal adalah loket tempat kita memberi instruksi kerja kepada komputer dengan tulisan, bukan klik. Seperti berbicara dengan petugas loket: kita menyebut satu permintaan, petugas mengerjakannya, lalu melapor balik.

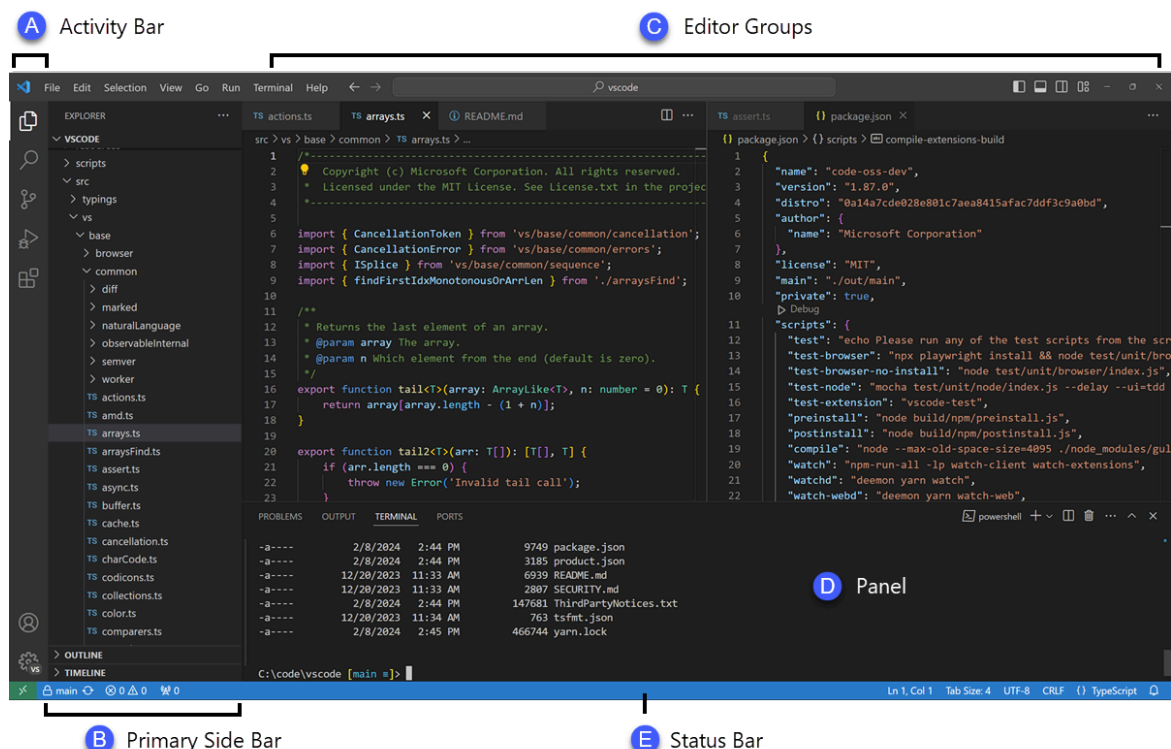
Buka **Visual Studio Code**, lalu buka direktori proyek kita lewat menu **File** → **Open Folder** → **anaLitika-perkotaan**.

💡 Pengetahuan VS Code

Mengenal Antarmuka Visual Studio Code

Saat pertama kali membuka Visual Studio Code, terdapat beberapa area utama yang esensial untuk menjalankan praktik kita:

1. **Activity Bar** (Sisi Paling Kiri): Berisi ikon-ikon pintasan untuk mengelola lembar kerja, seperti *Explorer* (melihat berkas proyek), *Search* (mencari teks), dan *Extensions* (memasang fitur tambahan).
2. **Side Bars** (Samping Activity Bar): Menampilkan daftar berkas dan folder aktif dalam direktori proyek Anda (*Explorer*).
3. **Editor Area/Editor Groups** (Bagian Tengah): Tempat utama untuk menulis dan menyunting berkas kode (.py atau .ipynb). Tempat tersebut bisa berjumlah lebih dari 1 dan mengelompokkan beberapa berkas, sehingga disebut “Groups”.
4. **Panel / Integrated Terminal** (Bagian Bawah): Area terminal terintegrasi tempat kita menjalankan perintah baris tanpa harus keluar dari VS Code.



Gambar 1: Tata Letak Antarmuka Visual Studio Code

Buka terminal bawaannya dengan menu **Terminal** → **New Terminal** — terminal ini otomatis berdiri di direktori proyek, jadi kita tidak perlu `cd` lagi. Pastikan papan nama (`analitika-perkotaan`) menyala; jika belum, jalankan lagi `conda activate analitika-perkotaan`.

Sekarang panggil Python:

```
python
```

Tampilan berubah: muncul keterangan versi Python dan tanda `>>>`. Tanda itu artinya Python siap menerima perintah **satu per satu** — mode ini disebut mode **interaktif** (atau *console*). Coba ketik dua perintah berikut, tekan Enter setiap selesai satu baris:

```
>>> print("Halo, Studio!")
Halo, Studio!
>>> 2 + 3
5
```

Python langsung menjawab setiap perintah, seperti petugas loket yang menjawab lisan pertanyaan lisan. Jika sudah puas, pamit dengan:

```
>>> exit()
```

Tanda `>>>` hilang dan kita kembali ke terminal biasa. Mode interaktif enak untuk coba-coba cepat, tetapi percakapannya **tidak tersimpan**. Untuk pekerjaan yang ingin kita simpan dan ulangi, kita butuh cara kedua: menuliskan perintah ke dalam berkas.

1.3.d Menulis Catatan Kerja: Membuat Skrip `sapa.py` dan Menjalankannya

Daripada mendiktekan perintah satu per satu, kita bisa menuliskan seluruh instruksi kerja di selembar kertas, lalu menyerahkannya ke loket untuk dikerjakan petugas dari atas sampai bawah. Lembar instruksi tertulis itu adalah **skrip**: berkas teks berisi perintah Python, disimpan dengan akhiran `.py`.

Di VS Code, buat berkas baru di dalam folder `skrip` dan beri nama `sapa.py`. Ketik isinya seperti berikut (silakan ganti nama sesuai nama Anda), lalu simpan:

```
nama_praktikan = "Aulia"
prodi = "Perencanaan Wilayah dan Kota"

print(f"Halo, {nama_praktikan} dari {prodi}!")
print("Selamat datang di praktikum Analitika Perkotaan.")
```

Untuk meminta petugas loket mengerjakannya, panggil `python` diikuti *path* menuju skripnya. Karena terminal kita berdiri di direktori proyek, *path* relatifnya adalah `skrip/sapa.py`:

```
python skrip/sapa.py
```

Sel berikut menjalankan isi `sapa.py` yang sama persis, supaya hasilnya terlihat langsung di modul ini:

```
nama_praktikan = "Aulia"  
prodi = "Perencanaan Wilayah dan Kota"  
  
print(f"Halo, {nama_praktikan} dari {prodi}!")  
print("Selamat datang di mata kuliah Analitika Perkotaan.")
```

```
Halo, Aulia dari Perencanaan Wilayah dan Kota!  
Selamat datang di mata kuliah Analitika Perkotaan.
```

Empat baris barusan memperkenalkan tiga hal paling mendasar dalam Python. Ketiganya dijelaskan pada *callout* di bawah — bacalah pelan-pelan di rumah, karena semuanya akan dipakai di **setiap** pertemuan setelah ini.

Konsep Python

Variabel dan Penugasan Nilai

`nama_praktikan = "Aulia"` artinya: siapkan sebuah kotak berlabel `nama_praktikan`, lalu masukkan tulisan `"Aulia"` ke dalamnya. Kotak berlabel itulah **variabel**, dan tanda `=` disebut **penugasan** (*assignment*) — perintah “isikan nilai di kanan ke dalam label di kiri”, bukan pernyataan matematika “sama dengan”.

Setelah diisi, label bisa dipakai di mana pun untuk memanggil isinya — itulah sebabnya `print(f"...{nama_praktikan}...")` bisa menampilkan `Aulia`. Isi kotak juga boleh diganti kapan saja dengan penugasan baru, misalnya `nama_praktikan = "Bima"`; isi lama tertimpa oleh yang baru.

Di studio, variabel seperti kotak berlabel di rak: sekali kotak “data survei” diisi, siapa pun yang butuh tinggal menyebut labelnya tanpa memindahkan isinya ke sana-sini. Nanti kita akan sering menaruh hal-hal panjang (misalnya *path* berkas data) ke dalam variabel pendek agar tidak lelah mengetik.

Konsep Python

Tipe Data Skalar — String dan Angka

Perhatikan perbedaan dua macam isi kotak: `"Aulia"` ditulis **di antara tanda kutip**, sedangkan angka seperti `5` atau `2026` ditulis polos. Tanda kutip adalah penanda bahwa isinya adalah **teks** — Python menyebutnya **string** (`str`). Tanpa tanda kutip, Python membacanya sebagai **angka**: bilangan bulat (**integer**/`int`, misalnya `2026`) atau bilangan berkoma (**float**, misalnya `197.22` — perhatikan pemisah desimalnya titik, bukan koma).

Jenis-jenis isi ini disebut **tipe data**, dan yang berisi satu nilai tunggal seperti di atas disebut tipe data **skalar**. Tipe menentukan apa yang bisa dilakukan: angka bisa dihitung (`2 + 3` menghasilkan `5`), sedangkan string dirangkai (`"20" + "26"` menghasilkan `"2026"`, bukan `46!`). Salah membedakan keduanya adalah sumber kekeliruan yang sangat umum — kita akan bertemu pesan kesalahannya sebentar lagi.

Konsep Python

print() dan f-string

`print()` adalah perintah untuk **menampilkan** sesuatu ke layar — petugas loket membacakan isi kotak yang kita sebut. Apa pun yang ditaruh di dalam tanda kurungnya akan dilaporkan ke terminal.

Bentuk `f"Halo, {nama_praktikan} dari {prodi}!"` disebut **f-string** (huruf `f` di depan tanda kutip artinya *formatted*). Di dalam f-string, bagian yang diapit kurung kurawal `{...}` akan **diganti dengan isi variabelnya** saat ditampilkan. Ini cara paling rapi untuk mencampur teks tetap dengan nilai variabel — tanpa f-string kita harus menyambung teks potong demi potong.

1.3.e Membaca Pesan Kegagalan: Tiga *Error* Paling Umum dan *Debugging* Dasar

Cepat atau lambat — biasanya cepat — kode kita akan gagal berjalan dan Python menampilkan tulisan merah yang panjang. Jangan panik: pesan itu bukan kemarahan, melainkan **petugas loket yang balik bertanya** karena instruksi kita tidak bisa dia pahami atau kerjakan. Seperti peserta studio yang menerima catatan revisi dari asisten studio, perencana yang baik tidak takut pada catatan revisi; programmer yang baik tidak takut pada pesan *error*.

Kunci membacanya: lihat **baris paling bawah** terlebih dahulu. Di sanalah tertulis **nama jenis kesalahannya** beserta keterangan singkat, sedangkan baris-baris di atasnya (disebut *traceback*) menunjukkan di berkas dan baris ke berapa kesalahan terjadi. Tiga jenis inilah yang paling sering menyapa pemula — kita sengaja memproduksi ketiganya sekarang, supaya saat muncul sungguhan kita sudah kenal wajahnya.

Wajah pertama: `SyntaxError` — kalimat instruksinya cacat. Python punya tata bahasa. Lupa menutup tanda kutip atau tanda kurung membuat kalimat kita tidak selesai, dan Python menolak bahkan untuk mulai bekerja. Jalankan sel berikut yang lupa menutup tanda kutip dan kurung:

```
print("Halo, Studio!
```

```
SyntaxError: unterminated string literal (detected at line 1) (501600097.py, line 1)
Cell In[2], line 1
    print("Halo, Studio!
```

```
SyntaxError: unterminated string literal (detected at line 1)
```

Perhatikan baris terakhirnya: nama kesalahannya `SyntaxError`, dan tanda `^` menunjuk lokasi Python kebingungan. Perbaikannya: lengkapi pasangan tanda yang hilang — setiap `"` dan `(` harus ada penutupnya.

Wajah kedua: `NameError` — menyebut label kotak yang tidak ada. Kesalahan ini muncul saat kita memanggil variabel yang belum pernah diisi, paling sering karena **salah ketik**. Di sel berikut, kita bermaksud memanggil `nama_praktikan` tetapi namanya kurang satu huruf:

```
print(nama_praktika)
```

```
NameError: name 'nama_praktika' is not defined
```

```

NameError                                Traceback (most recent call last)
Cell In[3], line 1
----> 1 print(nama_praktika)

NameError: name 'nama_praktika' is not defined

```

Python menjawab `NameError: name 'nama_praktika' is not defined` — “tidak ada kotak berlabel itu di rak”. Perbaikannya: cocokkan ejaan nama variabel dengan yang didefinisikan, huruf demi huruf (huruf besar-kecil dibedakan!). Pada Python versi terbaru, sesekali pesan ini malah berbaik hati menebak maksud kita dengan saran `Did you mean: ...?`.

Wajah ketiga: `TypeError` — menyuruh dua tipe data yang tak cocok bekerja sama. Ingat pembahasan tipe data tadi: string dan angka punya kemampuan berbeda. Menyambung teks dengan angka memakai `+` membuat Python bingung harus menghitung atau merangkai:

```
"Kota Bandar Lampung berusia " + 342
```

```

TypeError: can only concatenate str (not "int") to str
-----
TypeError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 "Kota Bandar Lampung berusia " + 342

TypeError: can only concatenate str (not "int") to str

```

Baris terakhirnya `TypeError: can only concatenate str (not "int") to str` — “saya hanya bisa menyambung string dengan string, bukan dengan bilangan bulat”. Perbaikannya yang paling rapi sudah kita kenal: bungkus dengan f-string, `print(f"Kota Bandar Lampung berusia {342}")`, dan Python otomatis mengubah angkanya menjadi teks.

Dari tiga pengenalan ini, lahirlah rutinitas ***debugging*** (mencari dan memperbaiki kesalahan) yang akan kita pakai sepanjang semester:

1. Baca **baris paling bawah**: apa nama jenis kesalahannya?
2. Lihat **nomor baris** yang ditunjuk *traceback*: di mana lokasinya?
3. Perbaiki satu hal itu saja, lalu **jalankan ulang**.
4. Masih gagal? Ulangi dari langkah 1. Setiap pesan baru adalah petunjuk baru.

1.3.f Membandingkan Dua Berkas: Mengapa Satu File Harus Lengkap-Mandiri

Sekarang eksperimen kecil yang hasilnya akan kita tagih lagi di Modul 2. Buat **dua** berkas baru di folder `skrip`.

Berkas pertama, `hitung_lengkap.py`, berisi perhitungan kepadatan penduduk Kota Bandar Lampung beserta definisi variabelnya:

```

luas_wilayah_km2 = 197.22
jumlah_penduduk = 1166066

```

```
kepadatan = jumlah_penduduk / luas_wilayah_km2
print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

Berkas kedua, `hitung_pincang.py`, berisi **hanya** perhitungannya — definisi variabelnya sengaja tidak ditulis:

```
kepadatan = jumlah_penduduk / luas_wilayah_km2
print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

Jalankan keduanya bergantian dari terminal:

```
python skrip/hitung_lengkap.py
python skrip/hitung_pincang.py
```

Hasilnya:

```
Kepadatan penduduk: 5913 jiwa per km persegi
Traceback (most recent call last):
  File "skrip/hitung_pincang.py", line 1, in <module>
    kepadatan = jumlah_penduduk / luas_wilayah_km2
                ^^^^^^^^^^^^^^^^^^^^^
NameError: name 'jumlah_penduduk' is not defined
```

Berkas pertama berhasil; berkas kedua gagal dengan `NameError` — wajah kedua yang baru saja kita kenal. Padahal `hitung_lengkap.py` baru saja dijalankan dan variabelnya baru saja diisi. Mengapa `hitung_pincang.py` tidak bisa ikut memakainya?

Karena **setiap kali python berkas.py dijalankan, komputer membuka “sesi kerja” yang benar-benar baru**: papan tulisnya kosong, raknya kosong, tidak ada ingatan apa pun dari eksekusi sebelumnya maupun dari berkas lain. Begitu skrip selesai, sesi itu ditutup dan seluruh isinya dibuang. Dua pelajaran penting:

1. **Satu berkas skrip harus lengkap-mandiri**: semua variabel yang dipakai harus didefinisikan di dalam berkas itu sendiri, seperti lembar instruksi kerja yang harus bisa dikerjakan petugas loket tanpa bertanya ke lembar lain.
2. Ingat baik-baik perilaku “tidak ada ingatan” ini — pada Modul 2 kita akan bertemu Jupyter Notebook yang perilakunya **justru sebaliknya**, dan perbedaan itulah sumber kebingungan mahasiswa yang paling umum.

1.3.g Menyiapkan Buku Lapangan: Membuat Jupyter Notebook dan Menyambungkan *Kernel*

Peralatan terakhir yang kita siapkan hari ini adalah **Jupyter Notebook** — buku catatan lapangan seorang perencana-analis. Di lapangan, satu buku catatan memuat tulisan pengamatan, angka pengukuran, dan sketsa dalam halaman yang sama. Notebook persis seperti itu: dalam satu berkas `.ipynb` kita bisa menulis narasi, kode Python, sekaligus melihat hasil dan grafiknya. Hampir seluruh praktikum kita mulai Modul 2 akan tinggal di dalam notebook.

Buat notebook pertama Anda:

1. Di VS Code, klik kanan folder `modul` → **New File** → beri nama `buku-lapangan.ipynb`. VS Code otomatis membuka tampilan notebook yang tersusun atas **sel-sel**.
2. Di pojok kanan atas, klik **Select Kernel** → **Python Environments** → pilih `analitika-perkotaan`.

Langkah kedua itu yang paling penting. **Kernel** adalah mesin Python yang menjalankan **sel-sel notebook** — dan kita baru saja menyambungkan notebook ke mesin dari *environment* buatan kita sendiri, lengkap dengan semua *library* yang tadi dipasang lewat `environment.yml`. Notebook tanpa kernel seperti buku lapangan tanpa surveyor: catatannya ada, tetapi tidak ada yang menghitung.

3. **Buktikan kernelnya benar.** Sama seperti papan nama terminal di langkah kelima tadi, nama kernel yang tampil juga baru sebatas klaim. Buat sel pertama berisi dua baris berikut, lalu jalankan dengan tombol ▶ di kiri sel (atau `Shift+Enter`):

```
import sys
print(sys.executable)
```

Path yang tercetak **harus** memuat potongan `envs\analitika-perkotaan`. Jika yang muncul lokasi lain, kembali ke langkah 2 dan pilih ulang kernelnya; bila tetap tidak berubah, laporkan ke dosen.

⚠ Nama Kernel Bisa Berbohong

Daftar kernel yang Anda lihat di VS Code tidak dibaca langsung dari daftar *environment* Conda. Ia dibaca dari berkas-berkas kecil bernama `kernel.json`, dan di dalam berkas itu **nama yang ditampilkan** disimpan terpisah dari **Python yang benar-benar dijalankan**. Keduanya bisa tidak cocok: sebuah kernel bertuliskan “Analitika Perkotaan” sepenuhnya sah menunjuk Python mana pun di komputer, termasuk Python lama dengan versi *library* yang berbeda.

Kasus semacam ini benar-benar pernah terjadi dalam penyusunan modul ini: sebuah kernel bernama `analitika-perkotaan` ternyata menjalankan Python lain dengan versi Pandas yang lebih tua, dan akibatnya baru diketahui setelah keluaran notebook dipenuhi peringatan yang tidak semestinya ada. Satu baris `print(sys.executable)` di awal akan menghemat berjam-jam penelusuran.

4. Ketik perintah berikut di sel kedua, lalu jalankan:

```
print("Buku lapangan siap!")
```

```
Buku lapangan siap!
```

Jika tulisan di atas muncul di bawah sel Anda, seluruh meja kerja digital kita sudah beres: direktori proyek tertata, *environment* terpasang, terminal sudah bisa diajak bicara, dan buku lapangan tersambung ke mesinnya. Bagaimana sel-sel notebook ini sesungguhnya bekerja — dan mengapa perilakunya berlawanan dengan dua berkas `.py` tadi — sengaja kita simpan sebagai pembuka Modul 2.

1.4 Latihan Mandiri

Kerjakan berurutan — alurnya sama persis dengan instruksi yang baru kita lalui. Simpan semua berkasnya, karena akan diperiksa di awal pertemuan berikutnya.

1. **Menulis skrip.** Buat berkas baru `skrip/identitas.py` yang mendefinisikan **tiga variabel**: `nama` (string), `nim` (angka), dan `kelas_studio` (string), lalu menampilkan ketiganya ke terminal dalam **satu**

kalimat utuh menggunakan `print()` dan f-string: "Izin memperkenalkan diri, saya <nama>, NIM <nim>, dari kelas studio <kelas_studio>. Mohon kerja samanya, ya!"

2. **Menjalankan dari terminal.** Jalankan skrip tersebut dari terminal yang berdiri di direktori proyek. Tuliskan (di komentar paling bawah berkas, diawali tanda `#`) perintah apa yang Anda pakai.
3. **Memproduksi tiga kegagalan.** Rusak skrip Anda secara sengaja, **satu kesalahan pada satu waktu** (perbaiki dulu sebelum membuat kesalahan berikutnya), sehingga ketiga *error* yang kita kenal muncul semua:
 - a. Hapus satu tanda kutip penutup → jalankan → amati **SyntaxError**.
 - b. Salah-ketikkan nama variabel di dalam `print()` → jalankan → amati **NameError**.
 - c. Ganti f-string dengan penyambungan langsung `"NIM saya " + nim` → jalankan → amati **TypeError**.

Untuk setiap kegagalan, catat di komentar: apa **nama error**-nya dan apa **baris terakhir** pesannya. Setelah selesai, kembalikan skrip ke kondisi benar.

4. **Menutup di buku lapangan.** Buka `modul/buku-lapangan.ipynb`, pastikan kernel `analitika-perkotaan` tersambung, lalu buat sel yang menampilkan kalimat identitas yang sama seperti langkah 1. Anda akan sadar harus **mendefinisikan ulang** ketiga variabelnya di dalam notebook — jelaskan dalam satu kalimat (di sel teks/markdown) mengapa variabel dari `identitas.py` tidak bisa langsung dipakai di sini.
5. **Membuktikan ruang kerja.** Dengan papan nama (`analitika-perkotaan`) menyala, jalankan lagi ketiga perintah pembuktian dari langkah kelima:

```
where.exe python
python -c "import sys; print(sys.prefix)"
python -c "import sys; print(sys.path)"
```

Tempelkan ketiga keluarannya sebagai komentar (diawali tanda `#`) di bagian paling bawah `skrip/identitas.py`. Lalu tuliskan, masih sebagai komentar, satu kalimat yang menjawab: "**Python mana yang sesungguhnya menjalankan kode saya, dan dari mana saya tahu?**" Sebutkan pula apakah ruang kerja Anda memenuhi ketiga syarat lulus atau tidak — jika tidak, sebutkan syarat nomor berapa yang gagal. Ini bukan sekadar anjuran: ruang kerja yang tidak terbukti akan menyulitkan Anda di setiap modul setelah ini.

2 Pengolahan Data Terstruktur dengan Pandas

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu mendemonstrasikan pengolahan data terstruktur dasar menggunakan Pandas, mengidentifikasi unit amatan dan unit analisis dari dataset perkotaan, serta memecahkan masalah kebingungan urutan eksekusi sel (**state** bersama) pada Jupyter Notebook.

2.1 Pendahuluan

<masukkan narasi yang menceritakan 'big why' mengelola data perkotaan yang terstruktur di sini>

2.2 Gambaran Umum Kasus

Pemerintah Kota Bandar Lampung hendak mengalokasikan bantuan pembangunan, dan pertanyaan pertamanya sederhana: kelurahan mana yang paling membutuhkan? Untuk menjawabnya, kita sebagai perencana memerlukan profil sosio-demografi setiap kelurahan — berapa kepadatan penduduknya, seberapa lengkap fasilitas pendidikan usia dininya, dan seterusnya. Bahan bakunya bersumber dari data **PODES 2021** (BPS) Kota Bandar Lampung. Data tersebut disimpan dalam format CSV.

Di studio, data seperti ini biasa kita rekap secara mandiri ke dalam tabel Excel saat melakukan kompilasi data. Kali ini kita berkenalan dengan **Pandas**, *library* Python untuk mengolah tabel rekapitulasi semacam itu. Namun sebelum menyentuh data, kita tuntaskan dulu janji Modul 1: **membuktikan cara kerja sel Jupyter Notebook** yang perilakunya berlawanan dengan dua berkas `.py` kita. Setelah jembatan itu terlewati, kita akan membaca rekap data CSV ke dalam Pandas, mengenali anatomi tabelnya, membedakan kelurahan sebagai *unit amatan* dari kecamatan sebagai *unit analisis*, merapikan tabel dengan membuat-menyusun-menghapus kolom, dan menutup dengan menyaring kelurahan berdasarkan kriteria untuk menemukan kandidat prioritas bantuan.

2.3 Instruksi

Buat notebook baru `modul/modul-02.ipynb` di direktori proyek Anda, sambungkan ke *kernel analitika-perkotaan*, dan kerjakan seluruh instruksi di dalamnya.

2.3.a Jembatan Jupyter: Membuktikan Perilaku Sel dan State Bersama

Di Modul 1, `hitung_pincang.py` gagal karena setiap eksekusi `python berkas.py` adalah sesi kerja baru yang tidak mengingat apa pun. Sekarang kita reproduksi kasus yang sama di notebook — tetapi kali ini “berkas pincang”-nya kita pecah menjadi **dua sel**. Jalankan sel pertama:

```
luas_wilayah_km2 = 197.22
jumlah_penduduk = 1166066
```

Lalu jalankan sel kedua, yang hanya berisi perhitungan — persis si `hitung_pincang.py`:

```
kepadatan = jumlah_penduduk / luas_wilayah_km2
print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

```
Kepadatan penduduk: 5913 jiwa per km persegi
```

Berhasil! Padahal sel kedua tidak mendefinisikan variabelnya sendiri. Inilah perbedaan mendasarnya: sel-sel notebook **bukan** berkas-berkas terpisah, melainkan potongan-potongan percakapan dalam **satu sesi yang sama** — seperti saat kita mengetik berurutan di mode interaktif `>>>` kemarin, hanya saja kini tiap potongannya tersimpan rapi.

Analogi studionya: **kernel** adalah **papan tulis studio** tempat kita menulis bersama. Di sel pertama, kita menuliskan dua angka di papan; di sel kedua kita tinggal menoleh ke papan yang sama untuk memakainya tanpa harus menulis ulang. Segala coretan yang telah kita tulis di papan — variabel yang pernah kita isi — akan tetap bertahan di sana dan siap kita gunakan di sel mana pun. Kondisi papan tulis pada satu waktu tertentu itulah yang disebut dengan `***state***` yang **dibagi bersama** oleh semua sel.

2.3.b Eksperimen Papan Tulis: Mengapa Urutan Eksekusi Sel Sangat Menentukan Hasil

Papan tulis bersama itu nyaman, tetapi menyimpan satu jebakan yang membingungkan hampir semua pemula. Mari kita masuk ke jebakannya dengan sadar. Jalankan sel berikut, yang mengganti isi variabel `luas` dengan angka yang jelas-jelas salah:

```
luas_wilayah_km2 = 20
```

Sekarang **kembalilah ke sel perhitungan kepadatan di atas dan jalankan ulang sel itu** (klik selnya, lalu `Shift+Enter`). Hasilnya melonjak menjadi lebih dari 58 ribu jiwa per km persegi — karena perhitungan menoleh ke papan tulis dan menemukan angka `20` yang baru. Di modul ini kita tuliskan ulang perhitungannya sebagai sel berikut agar lonjakannya terlihat:

```
kepadatan = jumlah_penduduk / luas_wilayah_km2
print(f"Kepadatan penduduk: {kepadatan:.0f} jiwa per km persegi")
```

```
Kepadatan penduduk: 58303 jiwa per km persegi
```

Perhatikan dua hal:

1. Hasil sebuah sel **tidak ditentukan oleh posisinya di halaman**, melainkan oleh **urutan eksekusinya**. Sel yang tampak lebih atas bisa saja dijalankan belakangan dan memakai nilai yang lebih baru.
2. Angka dalam tanda kurung siku di kiri sel (misalnya `[5]`) adalah **nomor antrean eksekusi**. Kalau nomornya melompat-lompat tidak berurutan dari atas ke bawah, hasil yang tampil bisa jadi berasal dari papan tulis yang keadaannya sudah berubah.

 **Penting!**

Papan Tulis Tidak Menghapus Dirinya Sendiri

Coretan di papan tulis studio akan bertahan sampai kita sendiri menghapusnya — meski kita sudah selesai bekerja. Begitu pula variabel di **kernel**: sekali terisi, ia bertahan dan memengaruhi sel mana pun yang dijalankan setelahnya, **meskipun sel yang mengisinya sudah kita hapus dari notebook**.

Karena itu, biasakan ritual studio ini: sebelum kita mengumpulkan atau mempercayai hasil akhir notebook kita, jalankan **Restart kernel & Run All** (hapus papan tulis, lalu kerjakan seluruh sel berurutan dari atas). Jika ada sel yang gagal, berarti notebook Anda belum lengkap-mandiri secara berurutan — persis prinsip berkas `.py` di Modul 1, kini dalam versi notebook.

Sebelum lanjut, kembalikan papan tulis ke angka yang benar:

```
luas_wilayah_km2 = 197.22
```

2.3.c Membuka Rekap Data: Membaca File CSV dengan Pandas

Sekarang kasus intinya. Data PODES kelurahan Bandar Lampung tersimpan sebagai berkas **CSV** (*comma-separated values*): berkas teks biasa yang tiap barisnya adalah satu catatan dan tiap kolomnya dipisahkan koma — bentuk paling sederhana dari **data terstruktur**, data yang tertata dalam baris dan kolom bermakna tetap. Baris pertamanya berisi nama-nama kolom. Kalau dibuka dengan Notepad, isinya kira-kira:

```
kode_prov,nama_prov,kode_kab,nama_kab,kode_kec,nama_kec,kode_des,nama_des,...
18,Lampung,71,Kota Bandar Lampung,10,Teluk Betung Barat,5,Bakung,...
```

Membaca teks seperti itu dengan mata tentu melelahkan. Di sinilah **Pandas** bekerja: ia memuat CSV menjadi tabel rekapitulasi yang rapi — persis lembar rekap kompilasi data studio di Excel — yang di Pandas disebut **DataFrame**.

Berkas kita berisi 34 kolom, padahal pertemuan ini hanya butuh sebagian. Maka sebelum membaca, kita siapkan dua variabel penampung: `path_data` untuk alamat berkasnya (ingat, notebook kita berada di folder `modul`, sehingga `path` relatifnya naik satu tingkat dulu dengan `..`) dan `kolom_pilihan` untuk daftar kolom yang ingin dibaca. Dengan begitu, perintah bacanya sendiri tetap pendek:

```
import pandas as pd

path_data = "../dataset-ganjil-2026-2027/bdl_kelurahan.csv"
kolom_pilihan = ["nama_kec", "nama_des", "status_daerah",
                 "kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta",
                 "jml_ra_ba_negeri", "jml_ra_ba_swasta"]

df = pd.read_csv(path_data, usecols=kolom_pilihan)
df.head()
```

	nama_kec	nama_des	status_daerah	jml_tk_negeri	\
0	Teluk Betung Barat	Bakung	1	0	

1	Teluk Betung Barat	Kuripan	1	0
2	Teluk Betung Barat	Negeri Olok Gading	1	0
3	Teluk Betung Barat	Sukarame Ii	2	0
4	Teluk Betung Barat	Batu Putuk	1	0

	jml_tk_swasta	jml_ra_ba_negeri	jml_ra_ba_swasta	kepadatan_pddk
0	2	0	0	6479
1	0	0	0	16321
2	1	0	0	7761
3	3	0	0	1514
4	1	0	0	2214

Tiga baris pekerjaan terjadi di sana. `import pandas as pd` memanggil *library* Pandas dari rak **environment** kita dan memberinya nama panggilan pendek `pd` — cukup sekali di awal notebook. `pd.read_csv(...)` membaca berkas pada `path_data`, mengambil hanya kolom-kolom dalam `kolom_pilihan` (lewat `usecols=`), lalu hasilnya kita tampung di variabel `df`. Terakhir, `df.head()` mengintip **lima baris pertama** tabel untuk memastikan bacaan kita benar.

Konsep Python

Tipe Data *list*

`kolom_pilihan` adalah sebuah *list*: tipe data yang menampung **banyak nilai sekaligus dalam satu kotak, berurutan**. Penulisannya memakai kurung siku, dengan koma sebagai pemisah: `["nama_kec", "nama_des", ...]`. Kalau variabel biasa adalah satu kotak berlabel, **list** adalah **rak berlabel yang isinya bernomor urut**.

Nomor urut itu disebut **indeks**, dan — ini kekhasan Python — **dihitung mulai dari 0**: `kolom_pilihan[0]` berisi `"nama_kec"`, `kolom_pilihan[1]` berisi `"nama_des"`, dan seterusnya. Panjang **list** bisa ditanyakan dengan `len(kolom_pilihan)`.

list akan sangat sering kita pakai untuk menampung daftar nama kolom seperti ini — daripada mengetik daftar panjang berulang-ulang di banyak perintah, kita tulis sekali di satu variabel lalu sebut labelnya.

2.3.d Mengenal Struktur Tabel: Anatomi Baris, Kolom, dan Tipe Data *DataFrame*

Rekap data yang baru tiba di studio selalu kita periksa dulu kelengkapannya sebelum dipakai. Untuk *DataFrame*, pemeriksaan pertamanya adalah `info()`:

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 126 entries, 0 to 125
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -

```

```

0  nama_kec      126 non-null  object
1  nama_des      126 non-null  object
2  status_daerah 126 non-null  int64
3  jml_tk_negeri 126 non-null  int64
4  jml_tk_swasta 126 non-null  int64
5  jml_ra_ba_negeri 126 non-null int64
6  jml_ra_ba_swasta 126 non-null int64
7  kepadatan_pddk 126 non-null  int64
dtypes: int64(6), object(2)
memory usage: 8.0+ KB

```

Laporan itu padat informasi. Bacalah dari atas:

- **126 entries** — tabel berisi **126 baris**, dan setiap baris adalah **satu kelurahan**. Di sinilah bedanya dengan tabel Excel bebas: dalam data terstruktur, satu baris selalu mewakili satu satuan pencatatan yang sama.
- Daftar kolom beserta **Non-Null Count** — berapa baris yang **terisi** di tiap kolom. Semuanya **126 non-null**, artinya tidak ada data kosong pada kolom-kolom pilihan kita (kemewahan yang tidak selalu terjadi; menangani data kosong adalah porsi Modul 4).
- **Dtype** — **tipe data** tiap kolom, saudara dari tipe data skalar di Modul 1: **int64** untuk bilangan bulat, **float64** untuk bilangan berkoma, dan **object** untuk teks/string seperti nama kecamatan.

Untuk kolom-kolom angka, kita bisa langsung meminta ringkasan statistiknya:

```
df.describe()
```

	status_daerah	jml_tk_negeri	jml_tk_swasta	jml_ra_ba_negeri	\
count	126.000000	126.000000	126.000000	126.0	
mean	1.015873	0.103175	2.595238	0.0	
std	0.125483	0.330560	2.075297	0.0	
min	1.000000	0.000000	0.000000	0.0	
25%	1.000000	0.000000	1.000000	0.0	
50%	1.000000	0.000000	2.000000	0.0	
75%	1.000000	0.000000	4.000000	0.0	
max	2.000000	2.000000	10.000000	0.0	

	jml_ra_ba_swasta	kepadatan_pddk
count	126.000000	126.000000
mean	0.269841	9431.825397
std	0.662271	8428.116609
min	0.000000	627.000000
25%	0.000000	3466.750000
50%	0.000000	6450.500000
75%	0.000000	12693.500000
max	4.000000	43907.000000

`describe()` menghitung sekaligus: jumlah data (`count`), rata-rata (`mean`), simpangan baku (`std`), nilai terkecil-terbesar (`min`, `max`), dan kuartil (25%, 50%, 75%). Dari satu tabel ini saja sudah terbaca, misalnya, betapa timpangnya kepadatan penduduk antar-kelurahan: ada kelurahan yang hanya ratusan jiwa per

km², ada yang lebih dari 40 ribu. Pendalaman statistik deskriptif semacam ini adalah menu utama Modul 4 — hari ini cukup berkenalan.

2.3.e Kelurahan vs Kecamatan: Membedakan Unit Amatan dan Unit Analisis

Perhatikan lagi: setiap baris tabel kita adalah satu **kelurahan** — pada level itulah BPS mencatat datanya. Dalam bahasa metodologi, kelurahan adalah **unit amatan** (*unit of observation*) dataset ini: satuan tempat data dicatat, seperti satu lembar kuesioner terisi untuk satu kelurahan.

Namun pertanyaan perencanaan tidak selalu dijawab pada level pencatatan. Ketika pimpinan bertanya “kecamatan mana yang paling minim fasilitas PAUD?”, kesimpulannya diambil pada level **kecamatan** — itulah **unit analisis** (*unit of analysis*): satuan tempat kesimpulan dibuat, seperti lembar rekapitulasi per wilayah yang merangkum tumpukan kuesioner. Unit amatan dan unit analisis boleh sama, boleh berbeda; yang fatal adalah tidak sadar sedang bekerja di level yang mana.

Mari kenali kedua level itu di data kita. Satu kolom di *DataFrame* dipanggil dengan `df["nama_kolom"]`, dan `nunique()` menghitung banyaknya **nilai unik** (nilai yang berbeda-beda) di dalamnya:

```
print(f"Jumlah kelurahan (unit amatan) : {df['nama_des'].nunique()}")
print(f"Jumlah kecamatan                : {df['nama_kec'].nunique()}")
```

```
Jumlah kelurahan (unit amatan) : 126
Jumlah kecamatan                : 20
```

126 kelurahan tersebar di 20 kecamatan. Lalu berapa kelurahan yang dinaungi masing-masing kecamatan? `value_counts()` menghitung **frekuensi kemunculan** tiap nilai pada sebuah kolom — alat andalan untuk meringkas **data kategorikal** (data berupa golongan/kategori, bukan angka hitungan):

```
df["nama_kec"].value_counts()
```

nama_kec	
Kemiling	9
Panjang	8
Tanjung Karang Barat	7
Sukabumi	7
Kedamaian	7
Rajabasa	7
Tanjung Karang Pusat	7
Kedaton	7
Sukarame	6
Labuhan Ratu	6
Way Halim	6
Telukbetung Timur	6
Enggal	6
Teluk Betung Utara	6
Teluk Betung Selatan	6
Langkapura	5
Tanjung Senang	5

```
Tanjung Karang Timur    5
Bumi Waras              5
Teluk Betung Barat      5
Name: count, dtype: int64
```

Tabel kecil itu sudah merupakan rekapitulasi pada unit analisis kecamatan: Teluk Betung Selatan menaungi paling banyak kelurahan, dan seterusnya. Simpan baik-baik `value_counts()` di ingatan — di Latihan Mandiri ia akan dipakai untuk meringkas kategori yang lebih bermakna sosial-ekonomi, dan di Modul 3 kita akan membuat rekapitulasi per kecamatan yang lebih kaya menggunakan SQL.

2.3.f Merapikan Tabel: Membuat, Menyusun Ulang, dan Menghapus Kolom

Fasilitas pendidikan usia dini di data kita terpecah menjadi empat kolom: TK negeri, TK swasta, RA/BA negeri, dan RA/BA swasta (RA/BA adalah *raudatul athfal/bustanul athfal*, PAUD di bawah Kemenag). Untuk pertanyaan alokasi bantuan, yang lebih berguna adalah **totalnya**. Di lembar rekap studio, kita akan menambahkan satu kolom jumlahan di ujung tabel — di Pandas pun sama: menugaskan nilai ke nama kolom yang belum ada akan **membuat kolom baru**:

```
df["jml_paud"] = df["jml_tk_negeri"] + df["jml_tk_swasta"] + df["jml_ra_ba_negeri"]
+ df["jml_ra_ba_swasta"]
df.head(3)
```

	nama_kec	nama_des	status_daerah	jml_tk_negeri	\
0	Teluk Betung Barat	Bakung	1	0	
1	Teluk Betung Barat	Kuripan	1	0	
2	Teluk Betung Barat	Negeri Olok Gading	1	0	

	jml_tk_swasta	jml_ra_ba_negeri	jml_ra_ba_swasta	kepadatan_pddk	jml_paud
0	2	0	0	6479	2
1	0	0	0	16321	0
2	1	0	0	7761	1

Perhatikan dua hal: penjumlahan antar-kolom bekerja **serentak untuk seluruh 126 baris** tanpa kita menghitung satu-satu, dan kolom baru `jml_paud` muncul di **ujung kanan** tabel. Posisi di ujung kanan itu kurang enak dibaca — kolom terpenting itu layak berdiri di dekat identitas kelurahan. Menyusun ulang posisi kolom caranya sederhana: sodorkan **list** berisi urutan kolom yang kita mau (pemakaian ulang konsep *list* barusan):

```
urutan_kolom = ["nama_kec", "nama_des", "status_daerah",
                "kepadatan_pddk", "jml_paud",
                "jml_tk_negeri", "jml_tk_swasta",
                "jml_ra_ba_negeri", "jml_ra_ba_swasta"]

df = df[urutan_kolom]
df.head(3)
```

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
0	Teluk Betung Barat	Bakung	1	6479	
1	Teluk Betung Barat	Kuripan	1	16321	
2	Teluk Betung Barat	Negeri Olok Gading	1	7761	

	jml_paud	jml_tk_negeri	jml_tk_swasta	jml_ra_ba_negeri	jml_ra_ba_swasta
0	2	0	2	0	0
1	0	0	0	0	0
2	1	0	1	0	0

Terakhir, setelah totalnya ada, empat kolom pecahannya tidak kita perlukan lagi hari ini. Membiarkan kolom tak terpakai membuat tabel lebar dan mata cepat lelah — seperti membiarkan kertas kalkir bekas menumpuk di meja gambar. Kita buang dengan `drop()`, lagi-lagi lewat variabel *list* agar rapi:

```
kolom_dibuang = ["jml_tk_negeri", "jml_tk_swasta", "jml_ra_ba_negeri",
                 "jml_ra_ba_swasta"]

df = df.drop(columns=kolom_dibuang)
df.head(3)
```

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
0	Teluk Betung Barat	Bakung	1	6479	
1	Teluk Betung Barat	Kuripan	1	16321	
2	Teluk Betung Barat	Negeri Olok Gading	1	7761	

	jml_paud
0	2
1	0
2	1

Terakhir, setelah totalnya ada, empat kolom pecahannya tidak kita perlukan lagi hari ini. Membiarkan kolom tak terpakai membuat tabel lebar dan mata cepat lelah — seperti membiarkan kertas kalkir bekas menumpuk di ruang kerja kita. Kita buang dengan `drop()`, lagi-lagi lewat variabel *list* agar rapi:

2.3.g Menyaring Baris: Filtrasi Data Kelurahan Berdasarkan Kriteria Tertentu

Tabel sudah rapi; kini saatnya mengajukan pertanyaan sang pemberi kasus: **kelurahan mana yang paling membutuhkan bantuan fasilitas PAUD?** Tafsiran pertama yang paling sederhana: kelurahan yang fasilitas PAUD-nya sangat sedikit — katakanlah satu atau tidak ada sama sekali.

Menyaring baris di Pandas polanya `df[kondisi]`: “dari tabel `df`, ambil baris-baris yang memenuhi kondisi”. Kondisinya kita tulis sebagai perbandingan atas sebuah kolom:

```
kelurahan_minim_paud = df[df["jml_paud"] <= 1]
print(f"Jumlah kelurahan dengan PAUD <= 1: {len(kelurahan_minim_paud)}")
kelurahan_minim_paud.head()
```

Jumlah kelurahan dengan PAUD <= 1: 37

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
1	Teluk Betung Barat	Kuripan	1	16321	
2	Teluk Betung Barat	Negeri Olok Gading	1	7761	
4	Teluk Betung Barat	Batu Putuk	1	2214	
8	Telukbetung Timur	Perwata	1	21630	
9	Telukbetung Timur	Way Tataan	1	687	

	jml_paud
1	0
2	1
4	1
8	0
9	0

Konsep Python

Operator Perbandingan

`df["jml_paud"] <= 1` memakai **operator perbandingan** — tanda yang membandingkan dua nilai dan menjawab benar (`True`) atau salah (`False`):

Operator	Arti	Contoh
<code>==</code>	sama dengan	<code>status == 1</code>
<code>!=</code>	tidak sama dengan	<code>nama != "Bakung"</code>
<code>></code>	lebih besar	<code>kepadatan > 10000</code>
<code><</code>	lebih kecil	<code>luas < 5</code>
<code>>=</code>	lebih besar atau sama	<code>jml_paud >= 2</code>
<code><=</code>	lebih kecil atau sama	<code>jml_paud <= 1</code>

Perhatikan: **sama dengan ditulis dua kali** (`==`), karena tanda `=` tunggal sudah dipakai untuk penugasan variabel (Modul 1). Menukar keduanya adalah salah ketik yang sangat umum.

Saat dipakai pada kolom **DataFrame**, perbandingan diuji **serentak ke seluruh baris**, menghasilkan deretan `True/False` sepanjang tabel — dan `df[...]` menahan hanya baris-baris yang `True`. Itulah mekanika di balik penyaringan barusan.

```
kriteria_padat = df["kepadatan_pddk"] > 10000
kriteria_minim = df["jml_paud"] <= 1

kelurahan_prioritas = df[kriteria_padat & kriteria_minim]
print(f"Jumlah kelurahan prioritas: {len(kelurahan_prioritas)}")
kelurahan_prioritas
```

Jumlah kelurahan prioritas: 18

	nama_kec	nama_des	status_daerah	kepadatan_pddk	\
1	Teluk Betung Barat	Kuripan	1	16321	
8	Telukbetung Timur	Perwata	1	21630	
10	Telukbetung Timur	Kota Karang Raya	1	23796	
11	Teluk Betung Selatan	Gedong Pakuon	1	13067	
12	Teluk Betung Selatan	Talang	1	19524	
14	Teluk Betung Selatan	Teluk Betung	1	25011	
31	Tanjung Karang Timur	Tanjung Agung	1	37717	
33	Tanjung Karang Timur	Sawah Lama	1	35775	
34	Tanjung Karang Timur	Sawah Brebes	1	26055	
51	Tanjung Karang Pusat	Kaliawi	1	25700	
54	Tanjung Karang Pusat	Kaliawi Persada	1	13159	
55	Enggal	Gunung Sari	1	13476	
56	Enggal	Tanjung Karang	1	13571	
86	Kedaton	Sukamenanti Baru	1	20233	
87	Kedaton	Penengahan	1	21014	
88	Kedaton	Penengahan Raya	1	11023	
120	Way Halim	Jagabaya I	1	19406	
123	Way Halim	Gunung Sulah	1	14823	

jml_paud
1
0
8
0
10
0
11
0
12
0
14
0
31
1
33
1
34
1
51
1
54
1
55
0
56
0
86
1
87
1
88
0
120
0
123
1

Konsep Python

Operator Boolean & dan |

Untuk menggabungkan dua kondisi, Python menyediakan **operator boolean** (dinamai dari aljabar *Boolean*, aljabar benar-salah):

- & berarti **dan** — hasilnya `True` hanya jika **kedua** kondisi benar.

- `|` berarti **atau** — hasilnya `True` jika **salah satu saja** kondisi benar.

Dalam Python murni kata kuncinya `and` dan `or`, tetapi **khusus untuk kolom Pandas kita wajib memakai lambang `&` dan `|`**, karena pengujian dilakukan serentak ke ratusan baris sekaligus. Satu aturan lagi: jika kondisinya ditulis langsung tanpa variabel penampung, tiap kondisi **harus dikurung**, misalnya `df[(df["kepadatan_pddk"] > 10000) & (df["jml_paud"] <= 1)]` — tanpa kurung, Python salah membaca urutannya. Menampung kondisi ke variabel seperti yang kita lakukan membuat kurung tak diperlukan dan kodenya jauh lebih terbaca.

2.4 Latihan Mandiri

Kerjakan di notebook baru `modul/latihan-02.ipynb` dengan *kernel* `analitika-perkotaan`. Alurnya sama persis dengan instruksi di atas; yang berganti hanya kolom yang dipakai — kali ini **pendidikan dasar** (SD dan SMP), ditambah satu kolom kategorikal.

1. **Jembatan sel.** Di dua sel terpisah: sel pertama mendefinisikan dua variabel angka apa saja versi Anda sendiri, sel kedua menghitung sesuatu dari keduanya. Buktikan keduanya berbagi papan tulis, lalu jalankan **Restart kernel & Run All** dan pastikan semuanya tetap berjalan berurutan.
2. **Membaca data.** Baca `bdL_kelurahan.csv` memakai variabel `path_data` dan kolom_pilihan yang berisi: `nama_kec`, `nama_des`, `status_daerah`, `kepadatan_pddk`, `lap_usaha_sebagian_warga`, `jml_sd_negeri`, `jml_sd_swasta`, `jml_smp_negeri`, `jml_smp_swasta`.
3. **Anatomi tabel.** Periksa dengan `info()` dan `describe()`. Sebutkan (di sel teks): berapa barisnya, kolom mana saja yang bertipe angka, dan adakah kolom yang datanya kosong.
4. **Unit amatan vs unit analisis.** Hitung banyaknya nilai unik kelurahan dan kecamatan, lalu buat rekapitulasi frekuensi **data kategorikal** `lap_usaha_sebagian_warga` (lapangan usaha sebagian besar warga) dengan `value_counts()`. Nilainya berupa kode kategori BPS — arti tiap kode dapat dilihat di kamus data PODES 2021 yang dibagikan dosen. Tuliskan satu kalimat: lapangan usaha kode apa yang paling banyak menjadi tumpuan warga kelurahan-kelurahan Bandar Lampung, dan pada unit apa (amatan atau analisis) kesimpulan kalimat Anda itu berdiri?
5. **Merapikan tabel.** Buat kolom baru `jml_dikdas` (jumlah fasilitas pendidikan dasar) dari penjumlahan empat kolom SD/SMP, susun ulang kolom sehingga `jml_dikdas` berdiri setelah `kepadatan_pddk`, lalu hapus empat kolom pecahannya memakai variabel `list`.
6. **Menyaring baris.** Temukan **kelurahan prioritas fasilitas pendidikan dasar**: kepadatan penduduk di atas **15.000 jiwa/km²** dan `jml_dikdas` paling banyak **2**. Berapa kelurahan yang lolos? Tutup notebook Anda dengan satu paragraf sel teks yang menafsirkan daftar tersebut sebagaimana laporan singkat kepada pimpinan.

3 Pengolahan Data Terstruktur dengan SQL (SQLite)

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu mengoperasikan basis data relasional SQLite menggunakan kueri SQL dasar, menerapkan operasi `JOIN` untuk menghubungkan tabel relasional kelurahan-kecamatan, dan mengintegrasikan kueri SQL ke dalam *DataFrame* Pandas.

3.1 Pendahuluan

<masukkan narasi yang menceritakan ‘big why’ mengelola basis data perkotaan yang relasional di sini>

3.2 Gambaran Umum Kasus

Dalam skala kota, menyimpan seluruh data dalam satu berkas tunggal menimbulkan redundansi: nama kecamatan yang sama ditulis berulang-ulang pada setiap baris kelurahannya, sehingga boros ruang dan rawan tidak konsisten ketika data dimutakhirkan. Apalagi di Indonesia pemekaran wilayah adalah isu yang sering diangkat (Fokus: Geliat Wacana Pemekaran Daerah — CNN Indonesia). Mengubah-ubah data tersebut dalam satu berkas tunggal hanya akan memperumit pembaruan.

Perencana yang mengelola data lintas instansi perlu cara penyimpanan yang lebih tertib: **basis data relasional**. Menggunakan data profil kelurahan Kota Bandar Lampung dari PODES 2021 (BPS), kita akan memindahkan data kelurahan ke dalam basis data SQLite yang telah memuat tabel kecamatan, mengajukan pertanyaan kependudukan dan fasilitas pendidikan menggunakan kueri SQL — dari penyaringan sederhana hingga penggabungan kelurahan dengan kecamatan induknya — lalu menarik hasilnya kembali ke Pandas untuk diolah lebih lanjut.

3.3 Instruksi

Buat notebook baru `modul/modul-03.ipynb` di direktori proyek Anda, sambungkan ke *kernel analitika-perkotaan*, dan kerjakan seluruh instruksi di dalamnya.

3.3.a Lemari Arsip Digital: Pengenalan SQLite dan Koneksi Database

Sebelum memulai praktik, pahami dulu tiga konsep dasarnya: basis data relasional, SQL, dan SQLite.

3.3.a.a Apa itu Basis Data Relasional?

Basis data relasional (*relational database*) adalah sistem penyimpanan data digital yang mengorganisasi data ke dalam kumpulan tabel dua dimensi (baris dan kolom) yang saling berhubungan — saling **berelasi** — satu sama lain. Model ini pertama kali dirancang oleh Edgar F. Codd pada tahun 1970, dengan tiga karakteristik utama:

- **Struktur tabular** — setiap tabel mewakili satu entitas tertentu; misalnya satu tabel khusus data kelurahan dan satu tabel khusus data kecamatan.
- **Kunci relasi (key)** — hubungan antar-tabel diikat oleh kolom pengenalan unik yang sama; tabel kelurahan terhubung ke tabel kecamatan melalui kolom kunci wilayah `kode_kec`.
- **Efisiensi dan integritas** — dengan memisahkan entitas ke tabel-tabel yang berelasi, kita menghindari redundansi (penulisan data berulang) dan menjaga data tetap konsisten ketika terjadi pemutakhiran wilayah.

Dalam analisis perkotaan, perencana sering bekerja dengan banyak tabel yang saling berhubungan — data kependudukan, fasilitas, hingga ekonomi lintas proyek dan lintas instansi. Dibandingkan menumpuk semuanya di berkas-berkas CSV terpisah, basis data relasional menawarkan penyimpanan yang efisien, kueri lintas tabel, dan penghindaran redundansi.

3.3.a.b Apa itu SQL?

SQL (*Structured Query Language*) adalah bahasa standar untuk berkomunikasi dengan basis data relasional. Jika basis data adalah **lemari arsip digital** studio PWK, maka SQL adalah bahasa perintah tertib yang kita gunakan untuk meminta petugas arsip melakukan tindakan tertentu — mengambil berkas kelurahan tertentu, mencatat berkas baru, atau merekap fasilitas umum per kecamatan.

Lalu di mana posisinya terhadap Pandas yang baru kita pelajari? Tabel berikut membandingkan keduanya *head-to-head* (diacu dari *SQL for Data Analysis*, Tanimura, 2021):

Dimensi Perbandingan	SQL	Python (Pandas)
Lokasi & beban komputer	Pengolahan dilakukan langsung oleh sistem basis data di pusat penyimpanan; komputer kita tidak terbebani saat datanya sangat besar.	Pengolahan dilakukan sepenuhnya di memori komputer kita sendiri; komputer bekerja lebih keras jika datanya sangat besar.
Kekakuan struktur data	Data harus ditata rapi sejak awal dalam tabel-tabel yang saling terhubung — seperti sekat permanen di laci arsip.	Bentuk data lebih fleksibel dan bebas diubah di tengah jalan, tetapi perintah kodenya lebih panjang dan detail.
Mekanisme perulangan data	Otomatis memproses seluruh baris sekaligus; tidak perlu perintah membaca baris demi baris.	Kadang perlu perintah khusus agar komputer membaca data baris demi baris secara berurutan.
Kelengkapan analisis	Sangat cepat untuk menyaring, menyatukan, dan merekap tabel, tetapi tidak bisa dipakai membangun model simulasi kota yang rumit.	Ekosistem alat bantunya melimpah: grafik, visualisasi peta, hingga model simulasi masa depan kota.

Keduanya bukan pesaing, melainkan pasangan kerja: SQL menyiapkan dan merekap data di lemari arsip, Pandas mengolah hasil rekapnya lebih lanjut. Persis alur itulah yang kita jalani pada modul ini.

3.3.a.c Mengenal SQLite

SQLite adalah basis data relasional yang unik karena bersifat **serverless**. Berdasarkan dokumentasi resmi SQLite.org, SQLite tidak memerlukan proses server latar belakang (*daemon*) ataupun koneksi jaringan: mesin SQLite adalah *library* yang berjalan langsung di dalam proses aplikasi kita (*in-process*). Seluruh isi basis data — berapa pun banyak tabelnya — disimpan dalam **satu file biner tunggal** di *disk* lokal, sehingga bersifat *zero-configuration*, portabel, dan mudah disalin antarfolder.

Di sinilah jawaban teka-teki penting modul ini: **satu file .sqlite adalah satu basis data utuh, bukan satu tabel**. Berbeda dengan CSV yang satu file-nya hanya memuat satu tabel, satu file .sqlite adalah satu lemari arsip lengkap yang lacinya (tabelnya) bisa lebih dari satu.

Pengetahuan Komputer

Basis Data Klien-Server vs *Serverless*

Basis data kelas industri seperti **PostgreSQL** bekerja dengan arsitektur **klien-server**: data dikelola oleh satu program server yang berjalan terus-menerus, dan analis mengaksesnya lewat jaringan memakai akun serta izin akses. Arsitektur ini unggul untuk data besar yang diakses banyak orang serentak — tetapi merepotkan untuk belajar karena menuntut instalasi dan konfigurasi server.

SQLite memangkas semua itu: tanpa server, tanpa akun, cukup satu file. Karena kesederhanaannya, SQLite menjadi basis data yang paling banyak terpasang di dunia — tertanam di ponsel, peramban, hingga berbagai aplikasi sehari-hari.

Satu klasifikasi lagi: SQLite dan PostgreSQL sama-sama berorientasi baris (*row-store*), menyimpan data per baris di *disk* — efisien untuk pencatatan dan pemutakhiran. Kontrasnya adalah basis data analitik berorientasi kolom (*column-store*) seperti BigQuery dan Snowflake, yang menyimpan data per kolom sehingga sangat cepat merekap dataset berskala raksasa.

Praktik kita mulai. Muat *library* `sqlite3` — pustaka standar bawaan Python, tanpa instalasi tambahan — bersama `Pandas`, lalu buka koneksi ke file `bd1.sqlite` dari direktori dataset yang dibagikan dosen:

```
import sqlite3
import pandas as pd

path_db = "../dataset-ganjil-2026-2027/bd1.sqlite"
koneksi = sqlite3.connect(path_db)
```

`sqlite3.connect(...)` membuka jalur komunikasi ke lemari arsip dan menampungnya di variabel `koneksi`. Semua perintah SQL kita selanjutnya akan lewat jalur ini.

Penting!

Path Salah Tidak Menimbulkan Galat

Jika *path* yang diberikan ke `sqlite3.connect(...)` keliru, Python **tidak akan protes** — ia justru membuat file basis data baru yang kosong di lokasi keliru tersebut. Maka bila inspeksi pada langkah berikutnya tidak menemukan tabel apa pun, jangan lanjut: periksa kembali `path_db` Anda, hapus file kosong yang telanjur terbuat, lalu ulangi.

3.3.a.d Inspeksi Isi Basis Data

Karena satu file dapat berisi banyak tabel, kebiasaan pertama saat menerima file SQLite adalah **membuka lemarinya dan membaca daftar lacinya**. SQLite menyediakan tabel sistem bernama `sqlite_master` (nama barunya: `sqlite_schema`) yang mencatat metadata seluruh objek di dalam basis data. Tiga kolom pentingnya: `type` (jenis objek: `table`, `index`, `view`, `trigger`), `name` (nama objek), dan `sql` (pernyataan `CREATE TABLE` asli pembuat objek itu).

Jalankan kueri pertama Anda untuk mendaftarkan semua tabel:

```
kueri_daftar_tabel = "SELECT name FROM sqlite_master WHERE type='table'"

daftar_tabel = koneksi.execute(kueri_daftar_tabel).fetchall()
print(daftar_tabel)
```

```
[('kecamatan',)]
```

Bacalah polanya: `koneksi.execute(...)` mengirim kueri ke basis data, dan `.fetchall()` mengambil seluruh baris hasilnya sebagai sebuah *list* (Modul 2) — setiap baris hasil tampil sebagai satu kelompok nilai di dalam tanda kurung. Hasilnya satu baris saja: lemari arsip kita baru berisi satu laci, tabel `kecamatan`.

Untuk membaca struktur kolom sebuah tabel, gunakan perintah sistem `PRAGMA table_info(nama_tabel)`. Ia mengembalikan rincian setiap kolom: nomor urut, nama kolom, tipe data, wajib-tidaknya terisi (`notnull`), nilai bawaan, dan status kunci primer (`pk`). Bentuk kueri standarnya juga tersedia: `SELECT * FROM pragma_table_info('nama_tabel')`.

```
struktur_kecamatan = koneksi.execute("PRAGMA table_info(kecamatan)").fetchall()
print(struktur_kecamatan)
```

```
[(0, 'kode_kec', 'INTEGER', 0, None, 0), (1, 'nama_kec', 'TEXT', 0, None, 0)]
```

Tabel `kecamatan` berisi dua kolom: `kode_kec` bertipe `INTEGER` (bilangan bulat) dan `nama_kec` bertipe `TEXT` (teks). Sekarang intip isinya — perintah `SELECT` akan kita bedah tuntas pada bagian Kueri Dasar; untuk sekarang cukup baca `SELECT * FROM kecamatan LIMIT 5` sebagai “ambilkan semua kolom dari tabel `kecamatan`, lima baris pertama saja”:

```
print(koneksi.execute("SELECT * FROM kecamatan LIMIT 5").fetchall())
```

```
[(10, 'Teluk Betung Barat'), (11, 'Telukbetung Timur'), (20, 'Teluk Betung Selatan'),
(21, 'Bumi Waras'), (30, 'Panjang')]
```

Kecamatan-kecamatan Kota Bandar Lampung tercatat rapi di laci ini, masing-masing dengan kode wilayah resmi BPS-nya.

💡 Pengetahuan VS Code

Inspeksi Visual dengan SQLite Explorer

File `.sqlite` adalah file biner — jika dibuka langsung di editor, isinya tampak seperti karakter acak. Agar dapat menjelajahnya secara visual, pasang ekstensi **SQLite** (penerbit: alexcvzz) dari panel Extensions VS Code.

Setelah terpasang: buka *Command Palette* (`Ctrl+Shift+P`), jalankan **SQLite: Open Database**, lalu pilih file `bd1.sqlite`. Panel **SQLITE EXPLORER** akan muncul di sisi kiri bawah; bentangkan untuk melihat daftar tabel, dan klik ikon panah di samping nama tabel untuk menampilkan isinya (*Show Table*).

Inilah padanan visual dari kueri `sqlite_master` dan `PRAGMA table_info` yang barusan Anda jalankan — dua cara membaca lemari arsip yang sama.

3.3.b Menyusun Laci Data: Membuat Tabel dan Mengisi Data dari Pandas

Laci kelurahan belum ada — kita yang akan menatanya sendiri dari berkas `bd1_kelurahan.csv` yang sudah kita kenal di Modul 2. Baca kembali berkas itu memakai pola `path_data` dan `kolom_pilihan`; kali ini kolom yang dibutuhkan adalah kunci wilayah, identitas kelurahan, kepadatan penduduk, dan fasilitas TK:

```
path_data = "../dataset-ganjil-2026-2027/bd1_kelurahan.csv"
kolom_pilihan = ["kode_kec", "nama_kec", "nama_des",
                 "kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta"]

df_kelurahan = pd.read_csv(path_data, usecols=kolom_pilihan)
df_kelurahan.head()
```

	kode_kec	nama_kec	nama_des	jml_tk_negeri	\
0	10	Teluk Betung Barat	Bakung	0	
1	10	Teluk Betung Barat	Kuripan	0	
2	10	Teluk Betung Barat	Negeri Olok Gading	0	
3	10	Teluk Betung Barat	Sukarame Ii	0	
4	10	Teluk Betung Barat	Batu Putuk	0	

	jml_tk_swasta	kepadatan_pddk
0	2	6479
1	0	16321
2	1	7761
3	3	1514
4	1	2214

Perhatikan kolom `nama_kec`: nama kecamatan yang sama tertulis berulang pada setiap baris kelurahannya. Buktikan skala redundansinya dengan dua alat Modul 2:

```
print(len(df_kelurahan))
print(df_kelurahan["nama_kec"].nunique())
```

126
20

Dari 126 baris, hanya ada 20 nama kecamatan unik — artinya setiap nama kecamatan ditulis berulang rata-rata enam kali. Padahal daftar resmi kecamatan sudah terstruktur rapi di tabel `kecamatan`. Menyimpan `nama_kec` sekali lagi di tabel kelurahan berarti memboroskan ruang dan membuka risiko ketidakonsistenan bila ada pemutakhiran nama.

Maka lakukan **normalisasi** sederhana: buang kolom `nama_kec` (cukup kunci `kode_kec` yang tinggal, sebagai pengikat relasi), lalu tulis `DataFrame`-nya menjadi tabel baru bernama `kelurahan` menggunakan `to_sql`:

```
df_kelurahan = df_kelurahan.drop(columns=["nama_kec"])
df_kelurahan.to_sql("kelurahan", koneksi, if_exists="replace", index=False)
```

126

Tiga hal terjadi pada perintah `to_sql` itu. Pertama, Pandas membuatkan tabel `kelurahan` di dalam basis data dan menyalin seluruh barisnya — angka yang tampil di bawah sel adalah jumlah baris yang tersalin. Kedua, `index=False` mencegah nomor indeks `DataFrame` ikut tersimpan sebagai kolom tersendiri. Ketiga, `if_exists="replace"` menentukan sikap ketika tabel bernama sama sudah ada: hapus dan tulis ulang.

 Penting!

Galat `table kelurahan already exists`

Tanpa `if_exists="replace"`, perilaku bawaan `to_sql` adalah `if_exists="fail"`: eksekusi kedua atas sel yang sama akan gagal dengan galat `ValueError: Table 'kelurahan' already exists`, karena lacinya memang sudah terisi. Ingat ritual **Restart kernel & Run All** dari Modul 2 — tanpa `replace`, notebook Anda tidak akan lolos ritual itu. Karena tabel ini milik kita dan sumber kebenarannya adalah berkas CSV, menghapus lalu menulis ulang adalah sikap yang tepat di sini.

3.3.b.a Verifikasi Hasil Pengisian Data

Buktikan bahwa lemari arsip kini berisi dua laci, lalu periksa struktur laci baru itu. Variabel `kueri_daftar_tabel` masih tersimpan di papan tulis *kernel* — pakai ulang saja:

```
print(koneksi.execute(kueri_daftar_tabel).fetchall())
print(koneksi.execute("PRAGMA table_info(kelurahan)").fetchall())
```

```
[('kecamatan',), ('kelurahan',)]
[(0, 'kode_kec', 'INTEGER', 0, None, 0), (1, 'nama_des', 'TEXT', 0, None, 0), (2,
'jml_tk_negeri', 'INTEGER', 0, None, 0), (3, 'jml_tk_swasta', 'INTEGER', 0, None,
0), (4, 'kepadatan_pddk', 'INTEGER', 0, None, 0)]
```

Dua tabel dalam satu file — persis seperti janji konsep di awal. Perhatikan pula tipe datanya: `to_sql` menerjemahkan tipe Pandas secara otomatis menjadi `INTEGER` untuk kolom bilangan bulat, `REAL` untuk

bilangan desimal, dan `TEXT` untuk teks. Silakan segarkan panel SQLite Explorer Anda (ikon *refresh*) — tabel `kelurahan` kini juga tampil di sana.

 Penting!

Basis Data Ini Dipakai Lagi di Modul-Modul Berikutnya

Lemari arsip `bd1.sqlite` yang baru Anda tata bukan sekadar latihan sekali pakai: modul-modul berikutnya akan **membuka kembali basis data ini** sebagai sumber data praktik. Jangan menghapus atau memindahkan file tersebut dari direktori dataset, dan pastikan tabel `kelurahan` benar-benar terbentuk (lolos verifikasi di atas) sebelum melanjutkan. Jika di kemudian hari file ini rusak atau terhapus, tabel `kelurahan` selalu dapat ditata ulang dari berkas CSV sumbernya dengan mengulang bagian ini.

3.3.c Kueri Dasar: Mengambil Data dengan `SELECT` dan Menyaring dengan `WHERE`

Lemari sudah tertata; saatnya belajar meminta petugas arsip bekerja. Kueri pengambilan data tersusun dari klausa-klausa: `SELECT` (kolom apa yang diambil), `FROM` (dari tabel mana), `WHERE` (baris mana yang lolos saringan), `ORDER BY` (diurutkan menurut apa; tambahkan `DESC` untuk urutan menurun), dan `LIMIT` (berapa baris maksimal yang dikembalikan). Klausa `WHERE` adalah padanan penyaringan baris Pandas pada Modul 2 — operator perbandingannya (`>`, `<=`, dan kawan-kawannya) bekerja serupa.

Dua bekal tambahan dari Tanimura (2021, Bab 2) untuk menghadapi data lapangan:

1. **Penanganan nilai kosong** — data yang hilang direpresentasikan sebagai `NULL`, dan menyaringnya memakai operator khusus `IS NULL` atau `IS NOT NULL` (bukan `= NULL`). Kolom-kolom `PODES` yang kita pakai kebetulan terisi lengkap, tetapi saringan ini adalah sabuk pengaman standar ketika bekerja dengan data instansi yang kelengkapannya belum tentu terjamin.
2. **Konversi tipe data** (*type casting*) — SQLite bertipe dinamis; untuk perhitungan yang konsisten kita bisa memaksa tipe tertentu secara eksplisit dengan `CAST(kolom AS tipe)`. Basis data lain seperti PostgreSQL menyediakan jalan pintas berupa operator `::`.

Susun kueri untuk menemukan kelurahan terpadat: kepadatan di atas 10.000 jiwa/km²,urut dari yang terpadat, sepuluh teratas. Kueri sepanjang ini tidak nyaman ditulis sebagai satu baris — tampung dalam string multi-baris:

```
kueri_dasar = """
SELECT
    nama_des,
    kode_kec,
    CAST(kepadatan_pddk AS REAL) AS kepadatan
FROM
    kelurahan
WHERE
    kepadatan_pddk IS NOT NULL
    AND kepadatan_pddk > 10000
ORDER BY
    kepadatan DESC
```

```
LIMIT 10
```

```
"""
```

```
hasil_dasar = koneksi.execute(kueri_dasar).fetchall()
print(hasil_dasar)
```

```
[('Kota Karang', 11, 43907.0), ('Tanjung Agung', 40, 37717.0), ('Sawah Lama', 40, 35775.0), ('Sukajawa', 70, 31830.0), ('Sukamenanti', 80, 28790.0), ('Kebon Jeruk', 40, 26592.0), ('Sawah Brebes', 40, 26055.0), ('Kaliawi', 60, 25700.0), ('Teluk Betung', 20, 25011.0), ('Kota Karang Raya', 11, 23796.0)]
```

Sepuluh kelurahan terpadat Bandar Lampung kini di tangan Anda. Klausula `AS` kepadatan pada `SELECT` memberi nama panggilan (*alias*) untuk kolom hasil — nama itulah yang dikenali `ORDER BY`.

Konsep Python

String Multi-baris (*Triple-Quoted*)

Di Modul 1, string selalu kita tulis satu baris dengan sepasang tanda kutip. Python menyediakan bentuk panjangnya: **string multi-baris**, diapit tiga tanda kutip (`"""` di awal dan di akhir). Segala yang berada di antaranya — ganti baris, indentasi, spasi — tersimpan apa adanya sebagai **satu nilai string biasa**.

Itulah sebabnya `kueri_dasar` dapat kita tata bertingkat agar enak dibaca tanpa menimbulkan galat sintaksis: bagi Python ia tetap satu string utuh yang tinggal dikirim ke basis data. Pola ini akan sering kembali setiap kali kita menampung teks panjang di dalam kode.

Pengetahuan Komputer

KAPITAL, Indentasi, dan Urutan Evaluasi

Bagi basis data, `select` dan `SELECT` sama saja. Penulisan kata kunci dengan huruf KAPITAL dan pemenggalan klausa dengan indentasi adalah **konvensi gaya** antarmanusia (Tanimura, 2021, Bab 8): mata cepat menangkap kerangka kueri, dan kueri panjang tetap terbaca.

Trivia keduanya lebih dalam: urutan penulisan klausa bukanlah urutan kerjanya. Secara internal, basis data mengevaluasi `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`. Jadi meski `SELECT` ditulis paling atas, ia justru dievaluasi belakangan — petugas arsip memilih laci dulu, menyaring berkasnya, baru memutuskan kolom mana yang disalin ke laporan.

3.3.d Penggabungan Wilayah: Menghubungkan Tabel Kelurahan dan Kecamatan dengan `JOIN`

Hasil kueri barusan hanya memuat `kode_kec` — angka yang tidak bercerita. Nama kecamatannya ada di laci sebelah. Operasi `JOIN` menyatukan berkas dari dua laci menggunakan pengikat kunci relasi: klausa `ON kel.kode_kec = kec.kode_kec` memerintahkan petugas arsip menjodohkan setiap baris kelurahan dengan baris kecamatan yang kode wilayahnya sama.

Dua perkakas penyerta `JOIN`:

- **Alias tabel** — kelurahan AS kel memberi nama panggilan pendek, sehingga penyebutan kolom lintas tabel (kel.nama_des, kec.nama_kec) tetap ringkas.
- **Empat jenis JOIN** — **INNER JOIN** hanya mengembalikan baris yang berjodoh di kedua tabel; **LEFT JOIN** mempertahankan seluruh baris tabel kiri walau tidak berjodoh; **RIGHT JOIN** kebalikannya; **FULL OUTER JOIN** mempertahankan keduanya. Kita memakai **LEFT JOIN** dengan tabel **kelurahan** di kiri: setiap kelurahan wajib tampil, berjodoh maupun tidak.

Satu kewaspadaan dari Tanimura (2021, Bab 2): **JOIN** yang kuncinya bermasalah dapat menggandakan baris hasil secara diam-diam. Kata kunci **DISTINCT** tepat setelah **SELECT** memerintahkan pembuangan baris hasil yang kembar persis — sekaligus menjadi alat pembuktian: bila jumlah baris hasil ber-**DISTINCT** tidak berkurang, tidak ada penggandaan yang terjadi.

```
kueri_gabung = """
SELECT DISTINCT
    kel.nama_des,
    kec.nama_kec,
    kel.kepadatan_pddk
FROM
    kelurahan AS kel
LEFT JOIN
    kecamatan AS kec ON kel.kode_kec = kec.kode_kec
ORDER BY
    kepadatan_pddk DESC
LIMIT 10
"""

hasil_gabung = koneksi.execute(kueri_gabung).fetchall()
print(hasil_gabung)
```

```
[('Kota Karang', 'Telukbetung Timur', 43907), ('Tanjung Agung', 'Tanjung Karang Timur', 37717), ('Sawah Lama', 'Tanjung Karang Timur', 35775), ('Sukajawa', 'Tanjung Karang Barat', 31830), ('Sukamenanti', 'Kedaton', 28790), ('Kebon Jeruk', 'Tanjung Karang Timur', 26592), ('Sawah Brebes', 'Tanjung Karang Timur', 26055), ('Kaliawi', 'Tanjung Karang Pusat', 25700), ('Teluk Betung', 'Teluk Betung Selatan', 25011), ('Kota Karang Raya', 'Telukbetung Timur', 23796)]
```

Sekarang setiap kelurahan terpadat tampil bersama nama kecamatan induknya — dua laci menyatu dalam satu laporan.

Pengetahuan Komputer

Kunci Primer dan Kunci Asing

Pada tabel **kecamatan**, kolom **kode_kec** berperan sebagai **kunci primer** (*primary key*): pengenalan unik yang nilainya pantang kembar — satu kode, satu kecamatan. Pada tabel **kelurahan**, kolom **kode_kec** yang sama berperan sebagai **kunci asing** (*foreign key*): penunjuk ke baris tabel lain yang boleh berulang — banyak kelurahan menunjuk satu kecamatan induk yang sama.

Mengapa mengikat relasi dengan kode, bukan dengan nama? Karena nama itu rapuh: ejaannya bisa berbeda antardokumen (misalnya “Teluk Betung” dan “Telukbetung”) dan namanya bisa berganti saat pemekaran wilayah, sedangkan kode wilayah resmi BPS dijaga konsisten. Inilah alasan kolom `nama_kec` aman kita buang dari tabel kelurahan: relasinya sudah dijamin oleh kunci.

3.3.e Agregasi Kota: Mengelompokkan Data dengan `GROUP BY`

Perencana jarang berhenti di daftar kelurahan; laporan akhirnya biasanya berupa rekap pada unit analisis kecamatan (Modul 2). Di SQL, rekap seperti itu dibuat dengan klausa `GROUP BY`, yang mengelompokkan baris-baris bernilai sama, lalu **fungsi agregasi** yang merangkum tiap kelompok menjadi satu angka: `AVG` (rata-rata), `SUM` (jumlah total), `COUNT` (cacah baris), `MIN`, dan `MAX`. `value_counts()` yang Anda pakai di Modul 2 sesungguhnya adalah `GROUP BY + COUNT` versi Pandas — dan inilah “rekapitulasi per kecamatan yang lebih kaya” yang dijanjikan modul lalu.

Aturan emasnya: **setiap kolom pada `SELECT` yang tidak dibungkus fungsi agregasi wajib dicantumkan di `GROUP BY`** — petugas arsip harus tahu persis menurut apa berkas dikelompokkan.

3.3.e.a Penanganan Nilai `NULL` dengan `COALESCE`

Satu jebakan aritmetika SQL: angka yang dijumlahkan dengan `NULL` hasilnya `NULL`. Bila salah satu dari `jml_tk_negeri + jml_tk_swasta` kosong, seluruh hasil penjumlahannya ikut kosong. Penangkalnya adalah fungsi `COALESCE(kolom, nilai_cadangan)` (Tanimura, 2021, Bab 2): ia memeriksa nilai dari kiri ke kanan dan mengembalikan yang pertama tidak-`NULL` — sehingga `COALESCE(jml_tk_negeri, 0)` berarti “pakai nilainya, atau 0 bila kosong”. Kolom TK kita memang terisi lengkap, tetapi membiasakan `COALESCE` pada penjumlahan lintas kolom adalah disiplin yang menyelamatkan ketika data instansi datang bolong-bolong.

Susun rekapnya: rata-rata jumlah TK (negeri + swasta) per kecamatan, beserta cacah kelurahannya,urut dari yang tertinggi:

```
kueri_rekap = """
SELECT
    kec.nama_kec,
    COUNT(keL.nama_des) AS jumlah_kelurahan,
    AVG(COALESCE(keL.jml_tk_negeri, 0) + COALESCE(keL.jml_tk_swasta, 0)) AS
rata_rata_tk
FROM
    kecamatan AS kec
LEFT JOIN
    kelurahan AS keL ON kec.kode_kec = keL.kode_kec
GROUP BY
    kec.nama_kec
ORDER BY
    rata_rata_tk DESC
"""

hasil_rekap = koneksi.execute(kueri_rekap).fetchall()
print(hasil_rekap)
```

```
[('Sukarame', 6, 5.833333333333333), ('Labuhan Ratu', 6, 4.166666666666667),
('Kedamaian', 7, 4.142857142857143), ('Rajabasa', 7, 3.857142857142857), ('Way
Halim', 6, 3.0), ('Sukabumi', 7, 2.857142857142857), ('Langkapura', 5, 2.6), ('Teluk
Betung Utara', 6, 2.5), ('Kemiling', 9, 2.4444444444444446), ('Tanjung Karang
Barat', 7, 2.4285714285714284), ('Kedaton', 7, 2.4285714285714284), ('Panjang', 8,
2.375), ('Teluk Betung Selatan', 6, 2.3333333333333335), ('Tanjung Karang Pusat', 7,
2.2857142857142856), ('Tanjung Senang', 5, 2.0), ('Tanjung Karang Timur', 5, 2.0),
('Telukbetung Timur', 6, 1.8333333333333333), ('Enggal', 6, 1.5), ('Teluk Betung
Barat', 5, 1.4), ('Bumi Waras', 5, 1.2)]
```

Perhatikan arah `JOIN`-nya kita balik — kecamatan di kiri — agar seluruh kecamatan pasti tampil di rekap, termasuk seandainya ada kecamatan yang tidak punya baris kelurahan.

Rekap 20 kecamatan itu benar isinya, tetapi bentuknya masih deretan baris polos hasil `fetchall()`: sulit dibaca, dan belum bisa diolah lanjut. Kita butuh jembatan kembali ke perkakas yang sudah kita kuasai.

3.3.f Jembatan SQL–Pandas: Membaca Hasil Kueri ke DataFrame Pandas

Jembatan itu bernama `pd.read_sql(kueri, koneksi)`: kirimkan kueri lewat koneksi, dan hasilnya langsung tersaji sebagai *DataFrame* — lengkap dengan nama kolom sesuai *alias* pada `SELECT`. Jalankan ulang `kueri_rekap` yang masih tersimpan di papan tulis *kernel*:

```
df_rekap = pd.read_sql(kueri_rekap, koneksi)
df_rekap.head(10)
```

	nama_kec	jumlah_kelurahan	rata_rata_tk
0	Sukarame	6	5.833333
1	Labuhan Ratu	6	4.166667
2	Kedamaian	7	4.142857
3	Rajabasa	7	3.857143
4	Way Halim	6	3.000000
5	Sukabumi	7	2.857143
6	Langkapura	5	2.600000
7	Teluk Betung Utara	6	2.500000
8	Kemiling	9	2.444444
9	Tanjung Karang Barat	7	2.428571

Bandingkan dengan keluaran `fetchall()` di atas: isinya sama, wujudnya berbeda — kini berupa tabel rapi yang siap disaring, direkap ulang, atau digabung dengan data lain memakai seluruh kemampuan Pandas dari Modul 2. Terbacalah alur lengkap modul ini: **CSV** → **to_sql** → **basis data SQLite** → **kueri SQL** → **read_sql** → **DataFrame**. Lemari arsip menertibkan penyimpanan; Pandas melanjutkan analisis.

3.3.f.a Menutup Sesi Kerja Basis Data

Tutup koneksi setiap kali selesai bekerja:

```
koneksi.close()
```

 Penting!

Tutup Koneksi Sebelum Pulang

Koneksi yang dibiarkan terbuka dapat mengunci file basis data — program lain (termasuk SQLite Explorer) bisa tertolak mengaksesnya, dan penulisan yang terputus di tengah berisiko merusak file. Ibaratnya laci lemari arsip: selesai bekerja, dorong lacinya sampai tertutup. Setelah `koneksi.close()`, variabel `koneksi` tidak dapat dipakai lagi; jika masih ingin mengueri, buka koneksi baru dengan `sqlite3.connect(...)`.

3.4 Latihan Mandiri

Kerjakan di notebook baru `modul/latihan-03.ipynb` dengan *kernel* `analitika-perkotaan`. Alurnya sama persis dengan instruksi di atas; yang berganti hanya fasilitasnya — dari TK ke **SMP**.

1. **Buka dan inspeksi.** Buka koneksi ke `bdl.sqlite` memakai variabel `path`, lalu daftar seluruh tabelnya dengan kueri `sqlite_master`.
2. **Tata ulang laci kelurahan.** Baca `bdl_kelurahan.csv` memakai `path_data` dan `kolom_pilihan` berisi: `kode_kec`, `nama_des`, `jml_smp_negeri`, `jml_smp_swasta`. Tulis ke tabel `kelurahan` dengan `to_sql` — pikirkan sendiri argumen `if_exists` yang tepat, mengingat tabel itu sudah ada.
3. **Saring dengan kueri dasar.** Dalam string multi-baris, susun kueri yang menampilkan kelurahan yang sama sekali belum memiliki SMP (jumlah SMP negeri ditambah swasta sama dengan nol) — amankan penjumlahannya dengan `COALESCE`.
4. **Rekap per kecamatan.** Susun kueri `LEFT JOIN + GROUP BY` untuk menghitung rata-rata jumlah SMP (negeri + swasta) per kecamatan,urut dari rata-rata **terendah** sehingga kecamatan yang paling minim fasilitas tampil teratas.
5. **Tarik ke Pandas.** Jalankan kueri rekap tersebut lewat `pd.read_sql`, tampung hasilnya di `DataFrame df_latihan`, lalu tampilkan.
6. **Interpretasi.** Di sel teks, tulis dua-tiga kalimat: kecamatan mana yang rata-rata SMP-nya paling rendah dan paling tinggi, dan apa maknanya bagi pemerataan fasilitas pendidikan dasar di Bandar Lampung.
7. **Tutup dan uji.** Tutup koneksi, lalu jalankan **Restart kernel & Run All** — seluruh sel harus lolos berurutan.

4 *Exploratory Data Analysis* (EDA)

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu melakukan *Exploratory Data Analysis* (EDA) secara statistik deskriptif pada data perkotaan, mendeteksi anomali data (*outlier*), menangani nilai kosong (*missing values*), dan menganalisis korelasi kasar antarvariabel perkotaan menggunakan Python.

4.1 Pendahuluan

<masukkan narasi yang menceritakan 'big why' mengeksplorasi karakter dan kualitas data perkotaan sebelum menganalisisnya di sini>

4.2 Gambaran Umum Kasus

Dalam pengerjaan studio, kompilasi data adalah tahap yang pantang dilewati: sebelum satu garis pun digambar ke laporan akhir, seluruh kuesioner kasar hasil survei lapangan diperiksa dulu satu per satu — adakah lembar yang kosong, adakah angka yang tidak masuk akal, dan pola awal apa yang mulai terlihat. Analisis yang dibangun di atas data yang belum diperiksa ibarat rencana tata ruang yang digambar di atas peta dasar yang salah.

Kebiasaan memeriksa itulah yang dalam analitika perkotaan disebut ***Exploratory Data Analysis* (EDA)**: eksplorasi awal untuk mengenali karakter dan menguji kualitas data sebelum data itu dipakai merumuskan apa pun. Menggunakan profil kelurahan Kota Bandar Lampung dari PODES 2021 (BPS) yang tersimpan dalam basis data hasil kerja kita di Modul 3, kita akan mengenali karakter data numerik lewat statistik deskriptif, membaca frekuensi data kategorikal, memburu nilai-nilai ekstrem (*outlier*) dengan metode IQR, menemukan dan menangani data kosong (*missing values*), memeriksa hubungan kasar antarvariabel lewat korelasi, lalu menutupnya dengan sintesis: seperti apa sesungguhnya karakter sosio-demografi kota ini menurut datanya sendiri.

4.3 Instruksi

Buat notebook baru `modul/modul-04.ipynb` di direktori proyek Anda, sambungkan ke *kernel analitika-perkotaan*, dan kerjakan seluruh instruksi di dalamnya.

4.3.a Mengenal Karakter Data: Statistik Deskriptif Dasar dengan Pandas

Pemeriksaan pertama seorang perencana terhadap data yang baru diterimanya selalu sama: mengenali karakternya secara ringkas. Untuk itu kita butuh datanya lebih dulu — dan kali ini sumbernya bukan lagi berkas CSV, melainkan **lemari arsip `bdl.sqlite` yang Anda tata sendiri di Modul 3**.

Ada satu pekerjaan penataan ulang: laci `kelurahan` yang Anda isi di Modul 3 baru memuat segelintir kolom fasilitas, padahal EDA menuntut **seluruh profil kelurahan** — topografi, kependudukan, hingga infrastruktur. Maka isi ulang laci itu dengan berkas lengkapnya: baca seluruh kolom `bdl_kelurahan.csv`, buang kolom redundan `nama_kec` (normalisasi Modul 3), lalu tulis ulang tabelnya dengan `to_sql`:

```
import sqlite3
import pandas as pd

path_db = "../dataset-ganjil-2026-2027/bdl.sqlite"
koneksi = sqlite3.connect(path_db)

path_data = "../dataset-ganjil-2026-2027/bdl_kelurahan.csv"
df_csv = pd.read_csv(path_data)
df_csv = df_csv.drop(columns=["nama_kec"])
df_csv.to_sql("kelurahan", koneksi, if_exists="replace", index=False)
```

126

Angka 126 menandakan seluruh baris kelurahan tersalin. Perhatikan bedanya dengan Modul 3: `pd.read_csv(path_data)` tanpa argumen `usecols` membaca **semua** kolom yang ada — itulah profil lengkap yang kita butuhkan.

⚠ Penting!

Jika Inspeksi Tidak Menemukan Tabel `kecamatan`

Langkah di atas menganggap `bdL.sqlite` Anda sehat: berisi tabel `kecamatan` bawaan kelas. Ingat perilaku `sqlite3.connect(...)` dari Modul 3 — *path* yang keliru tidak menimbulkan galat, melainkan diam-diam membuat basis data baru yang kosong. Bila kueri `JOIN` di bawah nanti menghasilkan kolom `nama_kec` yang kosong seluruhnya, berhenti: periksa `path_db` Anda, hapus file kosong yang telanjur terbuat, dan bila perlu ulangi bagian awal Modul 3.

Sekarang tarik profil lengkap itu ke *DataFrame*, sekaligus menjemput nama kecamatan induk dari laci sebelah dengan `LEFT JOIN` — persis pola Modul 3, ditampung dalam string multi-baris:

```
kueri_profil = """
SELECT
    kel.*,
    kec.nama_kec
FROM
    kelurahan AS kel
LEFT JOIN
    kecamatan AS kec ON kel.kode_kec = kec.kode_kec
"""

df = pd.read_sql(kueri_profil, koneksi)
print(df.shape)
```

(126, 34)

Satu notasi baru: `kel.*` pada `SELECT` berarti “seluruh kolom dari tabel `kelurahan`” — sehingga kita tidak perlu mengetik puluhan nama kolom satu per satu. Hasilnya 126 baris dan 34 kolom: seluruh profil kelurahan, plus `nama_kec` hasil `JOIN`.

Kini kenali karakternya. Metode `describe()` merangkum kolom-kolom numerik menjadi delapan angka: cacah data terisi (`count`), rata-rata (`mean`), simpangan baku (`std`), nilai terkecil (`min`), kuartil bawah (25%), median (50%), kuartil atas (75%), dan nilai terbesar (`max`). Tampung dulu kolom-kolom yang ingin diperiksa dalam variabel `list` (pola Modul 2) supaya daftar yang sama bisa dipakai ulang sepanjang modul:

```
kolom_numerik = ["kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta",
                 "jml_ra_ba_swasta"]

df[kolom_numerik].describe()
```

	kepadatan_pddk	jml_tk_negeri	jml_tk_swasta	jml_ra_ba_swasta
count	126.000000	126.000000	126.000000	126.000000
mean	9431.825397	0.103175	2.595238	0.269841
std	8428.116609	0.330560	2.075297	0.662271
min	627.000000	0.000000	0.000000	0.000000
25%	3466.750000	0.000000	1.000000	0.000000
50%	6450.500000	0.000000	2.000000	0.000000
75%	12693.500000	0.000000	4.000000	0.000000
max	43907.000000	2.000000	10.000000	4.000000

Bacalah tabel itu seperti perencanaan membaca rekap survei:

- **count = 126 di semua kolom** — tidak ada data kosong pada keempat kolom ini; kelengkapan kolom lain kita periksa belakangan.
- **kepadatan_pddk** — rata-ratanya sekitar 9.432 jiwa/km², tetapi simpangan bakunya hampir sama besar (± 8.428): sebaran antarkelurahan sangat lebar. Rentangnya membentang dari 627 sampai 43.907 jiwa/km², dan median (6.450) jauh di bawah rata-rata — tanda distribusinya **menceng**: segelintir kelurahan yang luar biasa padat menyeret rata-rata ke atas. Nilai-nilai ekstrem inilah yang akan kita buru pada bagian *outlier*.
- **jml_tk_negeri** — median bahkan kuartil atasnya 0: lebih dari tiga perempat kelurahan tidak memiliki satu pun TK negeri; layanan TK di kota ini nyaris sepenuhnya ditopang swasta.

Satu catatan penyaringan kolom: profil PODES juga memuat `jml_ra_ba_negeri`, tetapi seluruh 126 nilainya seragam 0 — kolom seragam tidak menyumbang informasi bagi statistik deskriptif, maka tidak kita ikutkan di `kolom_numerik`. Temuan “kolom seragam” semacam ini tetap layak dicatat; ia akan kembali di bagian sintesis.

4.3.b Data Kategorikal: Melihat Frekuensi dan Modus dengan `value_counts()`

Tidak semua kolom angka bermakna besaran. Pada profil PODES, banyak kolom berisi **kode kategori** — angkanya adalah label, bukan jumlah, sehingga merata-ratakannya tidak bermakna. Perlakuan yang tepat untuk data kategorikal adalah menghitung **frekuensi** tiap kategori dengan `value_counts()` (Modul 2), dan kategori terbanyaknya disebut **modus**. Periksa dua kolom: topografi wilayah (`topo`) dan status daerah saat pencacahan (`status_daerah`):

```
print(df["topo"].value_counts())
print(df["status_daerah"].value_counts())
```

```
topo
3    77
2    49
Name: count, dtype: int64
status_daerah
1    124
2     2
Name: count, dtype: int64
```

Arti tiap kode tercantum dalam kamus data PODES 2021 yang dibagikan dosen. Untuk `topo`: kode 3 adalah **dataran** dan kode 2 adalah **lembah/daerah aliran sungai** (kode 1, lereng/puncak, tidak muncul di data). Untuk `status_daerah`: kode 1 **perkotaan**, kode 2 **perdesaan**.

Terbaca dua hal. Pertama, modus topografi Bandar Lampung adalah dataran (77 kelurahan), tetapi porsi kelurahan lembah/DAS-nya besar juga (49 kelurahan) — hampir empat dari sepuluh. Kedua, yang lebih menarik: dua kelurahan di kota ini masih tercacah **perdesaan**. Temuan kecil semacam inilah yang sering luput bila kita tidak menyempatkan EDA. (Pandas juga menyediakan metode `mode()` untuk mengambil modus secara langsung; pada keluaran `value_counts()` modus selalu baris teratas, karena hasilnya terurut dari frekuensi terbesar.)

4.3.c Menemukan Keanehan: Deteksi *Outlier* (Nilai Ekstrem) Menggunakan Metode IQR

Statistik deskriptif tadi sudah membisikkan adanya nilai-nilai ekstrem. Sekarang kita tangkap mereka secara tegas dengan **metode IQR** (*Interquartile Range*), resep klasik John Tukey yang hanya membutuhkan dua angka dari tabel `describe()` tadi: kuartil bawah (Q1) dan kuartil atas (Q3). Jarak keduanya, $IQR = Q3 - Q1$, adalah rentang “separuh data yang di tengah”. Sebuah nilai dicap *outlier* bila jatuh di luar **pagar**: lebih kecil dari $Q1 - 1,5 \times IQR$, atau lebih besar dari $Q3 + 1,5 \times IQR$.

Hitung pagarnya untuk kepadatan penduduk — metode `quantile()` menerima posisi kuartil dalam pecahan (0.25 untuk Q1, 0.75 untuk Q3):

```
q1 = df["kepadatan_pddk"].quantile(0.25)
q3 = df["kepadatan_pddk"].quantile(0.75)
iqr = q3 - q1
batas_bawah = q1 - 1.5 * iqr
batas_atas = q3 + 1.5 * iqr

print(f"Q1 = {q1}, Q3 = {q3}, IQR = {iqr}")
print(f"Pagar bawah = {batas_bawah}, pagar atas = {batas_atas}")
```

```
Q1 = 3466.75, Q3 = 12693.5, IQR = 9226.75
Pagar bawah = -10373.375, pagar atas = 26533.625
```

Pagar bawahnya negatif — dan karena kepadatan penduduk mustahil negatif, otomatis tidak ada *outlier* di sisi bawah. Perburuan tinggal ke sisi atas: kelurahan mana yang kepadatannya menembus 26.533 jiwa/km²? Saring dengan filtrasi baris Modul 2:

```
outlier_kepadatan = df[(df["kepadatan_pddk"] < batas_bawah) | (df["kepadatan_pddk"] > batas_atas)]
outlier_kepadatan[["nama_des", "nama_kec", "kepadatan_pddk"]]
```

	nama_des	nama_kec	kepadatan_pddk
7	Kota Karang	Telukbetung Timur	43907
31	Tanjung Agung	Tanjung Karang Timur	37717
32	Kebon Jeruk	Tanjung Karang Timur	26592
33	Sawah Lama	Tanjung Karang Timur	35775
63	Sukajawa	Tanjung Karang Barat	31830
82	Sukamenanti	Kedaton	28790

Enam kelurahan tertangkap pagar, dipuncaki Kota Karang (Telukbetung Timur) dengan 43.907 jiwa/km² — hampir tujuh kali lipat median kota. Ingat: *outlier* bukan otomatis data yang salah. Kepadatan setinggi itu bisa jadi kenyataan permukiman pesisir yang memang sesak — justru karena itulah ia perlu ditandai: diperiksa kebenarannya, dan bila benar, diberi perhatian khusus dalam perencanaan.

Satu kolom sudah; tetapi memeriksa kolom-kolom lain dengan menyalin-tempel blok kode yang sama berulang kali jelas bukan cara kerja yang tertib. Serahkan pengulangannya kepada Python — jalankan pemeriksaan IQR untuk **setiap** kolom dalam `kolom_numerik` sekaligus, lengkap dengan penilaian otomatis atas hasilnya:

```
for kolom in kolom_numerik:
    q1 = df[kolom].quantile(0.25)
    q3 = df[kolom].quantile(0.75)
    iqr = q3 - q1
    batas_bawah = q1 - 1.5 * iqr
    batas_atas = q3 + 1.5 * iqr

    penanda_outlier = (df[kolom] < batas_bawah) | (df[kolom] > batas_atas)
    jumlah_outlier = penanda_outlier.sum()

    if jumlah_outlier == 0:
        keterangan = "aman"
    elif jumlah_outlier <= 5:
        keterangan = "perlu dicermati"
    else:
        keterangan = "perlu investigasi"

    print(f"{kolom}: {jumlah_outlier} kelurahan outlier - {keterangan}")
```

```
kepadatan_pddk: 6 kelurahan outlier - perlu investigasi
jml_tk_negeri: 12 kelurahan outlier - perlu investigasi
```

```
jml_tk_swasta: 1 kelurahan outlier - perlu dicermati
jml_ra_ba_swasta: 22 kelurahan outlier - perlu investigasi
```

Empat kolom diperiksa oleh satu blok kode. Dua hal kecil pada kode itu: `penanda_outlier` adalah deretan nilai benar/salah hasil filtrasi Modul 2, dan `.sum()` menghitung banyaknya `True` di dalamnya (Python menghitung `True` sebagai 1). Hasilnya patut direnungkan: `jml_ra_ba_swasta` dilaporkan punya 22 “outlier” dan `jml_tk_negeri` 12 — padahal kita tahu dari `describe()` bahwa mayoritas nilai kedua kolom itu 0, sehingga Q1 dan Q3-nya sama-sama 0, IQR-nya 0, dan **setiap nilai di atas nol otomatis dicap outlier**. Pelajarannya: keluaran metode adalah alarm awal, bukan vonis — penilaian akhir tetap di tangan perencana yang memahami konteks datanya.

Konsep Python

Perulangan `for` dan `range()`

Baris `for kolom in kolom_numerik:` memerintahkan Python: “untuk setiap isi *list* `kolom_numerik`, jalankan blok di bawah ini satu kali.” Pada setiap putaran, variabel `kolom` bergiliran menampung satu isi *list* — mula-mula “`kepadatan_pddk`”, lalu “`jml_tk_negeri`”, dan seterusnya sampai isi *list* habis. Yang termasuk “blok di bawah ini” adalah semua baris yang **berindentasi** (menjorok) di bawah baris `for` — begitu indentasi berakhir, berakhir pula wilayah pengulangan.

Ini persis ritme kompilasi data di studio: satu daftar periksa yang sama diterapkan ke setiap kuesioner dalam tumpukan, kuesioner demi kuesioner, sampai tumpukan habis — tanpa menulis ulang daftar periksanya untuk tiap lembar.

Bila yang dibutuhkan bukan “untuk setiap isi *list*” melainkan “ulangi sekian kali”, pasangkan `for` dengan `range()`: `for i in range(4):` mengulang blok empat putaran, dengan `i` bernilai 0, 1, 2, 3 secara bergiliran (ingat, Python menghitung dari nol). Bentuk inilah yang akan sering muncul ketika kita menjelajah struktur data yang lebih dalam di modul-modul lanjut.

Konsep Python

Percabangan `if-elif-else`

Blok `if-elif-else` membuat kode **memilih jalan**: Python memeriksa kondisi satu per satu dari atas — kondisi `if` dulu, lalu tiap `elif` (singkatan *else if*) — dan begitu menemukan kondisi pertama yang benar, hanya blok cabang itulah yang dijalankan; sisanya dilewati. Cabang `else` adalah penampung terakhir bila tidak ada satu pun kondisi yang benar. Kondisinya sendiri dibangun dari perkakas yang sudah Anda kuasai di Modul 2: operator perbandingan (`==`, `<=`, `>`) dan operator boolean.

Analoginya adalah momen menyortir kuesioner di studio: tiap lembar hasil survei Anda periksa dengan urutan pertanyaan yang tetap — terisi lengkap? masukkan ke tumpukan siap-olah; kurang satu-dua isian? beri catatan perbaikan; rusak parah? sisihkan untuk survei ulang. Satu lembar hanya berakhir di satu tumpukan, sebagaimana satu putaran percabangan hanya menjalankan satu cabang.

Perhatikan pula bahwa percabangan tadi bekerja **di dalam** perulangan: penilaian yang sama diterapkan pada setiap kolom yang digilir `for`. Memadukan keduanya — mengulang sambil memilih — adalah pola alur kendali yang akan terus Anda pakai sampai akhir buku ini.

4.3.d Mengisi Lembar Kosong: Identifikasi dan Penanganan *Missing Values*

Setelah nilai yang “terlalu ada”, giliran nilai yang **tidak ada**. Dalam *DataFrame*, isian yang kosong ditandai `NaN` (Not a Number), dan metode `isna()` memeriksanya sel demi sel. Rangkakan dengan `.sum()` untuk merekap jumlah kekosongan per kolom, lalu — karena kolomnya 34 — saring rekap itu agar hanya kolom yang benar-benar bermasalah yang tampil:

```
jumlah_kosong = df.isna().sum()
jumlah_kosong[jumlah_kosong > 0]
```

```
dm_permukiman_sutet    119
jml_bangunan_sutet     123
dtype: int64
```

Dari 34 kolom, hanya dua yang berlubang — tetapi lubangnya menganga: `dm_permukiman_sutet` kosong pada 119 baris dan `jml_bangunan_sutet` pada 123 baris, dari total 126. Jembatan kecil ke Modul 3: `NaN` di Pandas adalah kembaran `NULL` di SQL — kekosongan yang sama yang dulu kita saring dengan `IS NULL`. Buktikan kesetaraannya dengan bertanya langsung ke basis data:

```
kueri_kosong = """
SELECT
    COUNT(*) AS jumlah_baris_kosong
FROM
    kelurahan
WHERE
    jml_bangunan_sutet IS NULL
"""

pd.read_sql(kueri_kosong, koneksi)
```

	jumlah_baris_kosong
0	123

Angka yang sama, 123 — dua kaca mata, satu kenyataan.

Sebelum menambal, **diagnosis dulu penyebab kosongnya**. Ketiga kolom ini berkaitan: `dm_dilalui_sutet` mencatat apakah wilayah kelurahan dilalui jaringan listrik tegangan tinggi (SUTET/SUTT/SUTTAS; kode 1 ada, kode 2 tidak — lihat kamus data PODES 2021), `dm_permukiman_sutet` mencatat ada-tidaknya permukiman di bawah jaringan itu, dan `jml_bangunan_sutet` menghitung bangunannya. Pertanyaan lanjutan tentu hanya diisi bila jawaban pertanyaan awalnya “ada”. Buktikan dengan filtrasi silang:

```
df[df["dm_dilalui_sutet"] == 1][["nama_des", "dm_permukiman_sutet",
    "jml_bangunan_sutet"]]
```

	nama_des	dm_permukiman_sutet	jml_bangunan_sutet
4	Batu Putuk	2.0	NaN

	nama_des	dm_permukiman_sutet	jml_bangunan_sutet
11	Gedong Pakuon	2.0	NaN
22	Srengsem	2.0	NaN
68	Sumber Agung	1.0	12.0
109	Korpri Jaya	1.0	4.0
111	Korpri Raya	1.0	25.0
112	Sukarame Baru	2.0	NaN

Tepat 7 kelurahan dilalui SUTET — dan hanya pada merekalah `dm_permukiman_sutet` terisi; `jml_bangunan_sutet` bahkan hanya terisi pada 3 kelurahan yang permukimannya memang ada di bawah jaringan (kode 1). Kekosongan seperti ini disebut **kosong struktural**: bukan petugas lupa mencatat, melainkan pertanyaannya memang tidak berlaku bagi 119 kelurahan lain. Diagnosis ini menentukan resep penanganannya.

Pandas menyediakan dua penanganan pabrik: `dropna()` membuang setiap baris yang memuat kekosongan, dan `fillna(nilai)` menambal kekosongan dengan nilai tertentu. Coba dulu yang pertama — tetapi periksa dampaknya lewat `.shape` sebelum benar-benar memakainya:

```
print(df.shape)
print(df.dropna().shape)
```

```
(126, 34)
(3, 34)
```

⚠ Penting!

`dropna()` Bukan Jawaban Otomatis

Membuang semua baris yang berlubang menyisakan **3 kelurahan dari 126** — kita kehilangan 97% kota hanya karena dua kolom yang kosongnya pun struktural. Baris di atas sengaja tidak menimpa `df: df.dropna()` menghasilkan *DataFrame* baru, dan selama hasilnya tidak ditugaskan kembali ke variabel, data asli tetap utuh. Jadikan `.shape` kebiasaan wajib sebelum memutuskan membuang baris — dan buang hanya bila kekosongannya sedikit serta barisnya memang tidak terpakai.

Untuk kasus kita, resep yang masuk akal justru menambal. Pada `jml_bangunan_sutet`, kosong struktural bermakna pasti: tidak dilalui SUTET, atau tidak ada permukiman di bawahnya — artinya **nol bangunan**. Tambal dengan `fillna(0)`:

```
df["jml_bangunan_sutet"] = df["jml_bangunan_sutet"].fillna(0)

print(df["jml_bangunan_sutet"].isna().sum())
```

```
0
```

Kekosongannya tuntas. Adapun `dm_permukiman_sutet` kita **biarkan kosong**: ia kolom kode kategori (1/2), dan menambalnya dengan angka karangan justru memalsukan jawaban “pertanyaan ini tidak

berlaku". Tidak semua lubang harus ditambal — yang wajib adalah *mengetahui* lubangnya dan *sadar* memutuskan perlakuannya.

Rumuskan keputusan itu secara tertib untuk ketiga kolom SUTET sekaligus — perulangan dan percabangan yang baru Anda pelajari kini bekerja berpasangan:

```
kolom_sutet = ["dm_dilalui_sutet", "dm_permukiman_sutet", "jml_bangunan_sutet"]

for kolom in kolom_sutet:
    persen_kosong = df[kolom].isna().sum() / len(df) * 100

    if persen_kosong == 0:
        keputusan = "lengkap, siap dianalisis"
    elif persen_kosong < 50:
        keputusan = "tambal dengan fillna"
    else:
        keputusan = "periksa: kemungkinan kosong struktural"

    print(f"{kolom}: {persen_kosong:.1f}% kosong - {keputusan}")
```

```
dm_dilalui_sutet: 0.0% kosong - lengkap, siap dianalisis
dm_permukiman_sutet: 94.4% kosong - periksa: kemungkinan kosong struktural
jml_bangunan_sutet: 0.0% kosong - lengkap, siap dianalisis
```

Perhatikan `jml_bangunan_sutet` kini dilaporkan lengkap — buah dari tambalan barusan. (Notasi `:.1f` di dalam f-string Modul 1 sekadar merapikan tampilan angka menjadi satu digit di belakang koma.)

4.3.e Hubungan Awal: Analisis Korelasi Kasar Antarvariabel Numerik

Karakter tiap kolom sudah dikenali satu per satu; pemeriksaan pamungkas EDA adalah melihat **hubungan antar-kolom**. Ukuran kasarnya adalah **koefisien korelasi** (r): angka antara -1 dan $+1$ yang menunjukkan arah dan kekuatan hubungan garis lurus dua variabel — mendekati $+1$ berarti keduanya cenderung naik bersama, mendekati -1 berarti yang satu naik ketika yang lain turun, dan di sekitar 0 berarti nyaris tak ada pola bersama. Metode `corr()` menghitungnya untuk semua pasangan kolom sekaligus. Ujilah dugaan yang wajar bagi perencana: *makin padat sebuah kelurahan, makin banyak pula fasilitas pendidikannya?*

```
kolom_korelasi = ["kepadatan_pddk", "jml_tk_negeri", "jml_tk_swasta",
                  "jml_sd_negeri", "jml_sd_swasta"]

df[kolom_korelasi].corr()
```

	kepadatan_pddk	jml_tk_negeri	jml_tk_swasta	jml_sd_negeri	jml_sd_swasta
kepadatan_pddk	1.000000	-0.073817	-0.192438	0.155989	-0.041028
jml_tk_negeri	-0.073817	1.000000	0.003054	-0.061323	-0.102833
jml_tk_swasta	-0.192438	0.003054	1.000000	0.034594	0.536261

	kepadatan_pddk	jml_tk_negeri	jml_tk_swasta	jml_sd_negeri	jml_sd_swasta
jml_sd_negeri	0.155989	-0.061323	0.034594	1.000000	-0.074743
jml_sd_swasta	-0.041028	-0.102833	0.536261	-0.074743	1.000000

Cara membaca **matriks korelasi** ini: tiap sel adalah r untuk pasangan kolom baris-kolomnya; diagonalnya selalu 1 (setiap variabel berkorelasi sempurna dengan dirinya sendiri); dan matriksnya simetris — cukup baca separuhnya.

Hasilnya menampar dugaan kita. Korelasi kepadatan penduduk dengan keempat kolom fasilitas semuanya lemah (dari $-0,192$ sampai $0,156$): pada data ini, **kelurahan yang lebih padat tidak serta-merta lebih kaya fasilitas pendidikan** — sinyal awal yang layak menjadi bahan diskusi pemerataan layanan. Justru pasangan terkuat muncul di tempat lain: `jml_tk_swasta` dengan `jml_sd_swasta` ($r = 0,536$, kekuatan sedang) — kelurahan yang banyak TK swastanya cenderung banyak pula SD swastanya, gelagat bahwa penyedia layanan swasta berkerumun di kantong pasar yang sama.

Penting!

Korelasi Bukan Sebab-Akibat

Sekuat apa pun r , korelasi hanya mencatat dua hal bergerak bersama — ia tidak pernah membuktikan yang satu *menyebabkan* yang lain. Analisis kita di sini juga sengaja disebut korelasi **kasar**: belum ada uji signifikansi atau kendali atas variabel lain; perkakas statistika yang lebih ketat untuk itu Anda pelajari di mata kuliah statistika. Dalam EDA, posisi korelasi adalah pemandu — penunjuk hubungan mana yang pantas diselidiki lebih dalam, bukan hakim yang memutus perkara.

4.3.f Sintesis EDA: Menarik Kesimpulan Karakteristik Sosio-Demografi Kota

Pemeriksaan selesai; saatnya menyusun lembar kompilasi versi perencanaan. Digabungkan, temuan-temuan EDA kita membentuk potret karakter sosio-demografi Kota Bandar Lampung menurut datanya sendiri:

- **Kepadatan yang timpang.** Separuh kelurahan berkepadatan di bawah 6.450 jiwa/km², tetapi ekor distribusinya panjang ke atas hingga 43.907 — enam kelurahan ekstrem (terpadat: Kota Karang) menyeret rata-rata kota jauh di atas mediannya.
- **Kota di atas dataran dan lembah.** Modus topografi adalah dataran (77 kelurahan), disusul lembah/DAS (49); dan dua kelurahan ternyata masih tercacah berstatus perdesaan.
- **Layanan pendidikan usia dini bertumpu pada swasta.** Lebih dari tiga perempat kelurahan tanpa TK negeri, dan RA/BA negeri tidak ada sama sekali (kolom seragam nol).
- **Kekosongan data yang bermakna.** Dua kolom SUTET kosong secara struktural — kosongnya adalah informasi (“tidak berlaku”), bukan kelalaian; satu kolom kita tambal nol, satu lagi sengaja dibiarkan.
- **Kepadatan tidak menjamin fasilitas.** Korelasi kepadatan-fasilitas pendidikan lemah; yang menonjol justru kerumunan sesama layanan swasta (TK-SD swasta, $r = 0,536$).

Potret inilah bekal kita melangkah: dugaan mana yang pantas diuji serius, kolom mana yang siap pakai, dan angka mana yang harus diperlakukan hati-hati. Alur perkakasnya pun kini utuh — basis data menertibkan penyimpanan (Modul 3), Pandas mengolah (Modul 2), EDA mengenali karakter dan

kualitasnya (modul ini) — dan pada Modul 5, temuan-temuan yang masih berupa angka ini akan kita bunyikan menjadi **visualisasi**.

Tutup sesi kerja basis data Anda:

```
koneksi.close()
```

4.4 Latihan Mandiri

Kerjakan di notebook baru `modul/latihan-04.ipynb` dengan *kernel* `analitika-perkotaan`. Alurnya sama persis dengan instruksi di atas; yang berganti adalah fasilitasnya — dari TK ke **pendidikan dasar (SD dan SMP)** — plus satu kolom kategorikal yang lebih kaya.

1. **Buka lemari arsip.** Buka koneksi ke `bd.sqlite`, lalu tarik profil lengkap kelurahan ber-nama-`kec` ke *DataFrame* memakai `read_sql` dengan kueri `LEFT JOIN` seperti pada instruksi.
2. **Kolom total.** Buat dua kolom baru (Modul 2): `jml_sd` dari `jml_sd_negeri + jml_sd_swasta`, dan `jml_smp` dari `jml_smp_negeri + jml_smp_swasta`.
3. **Karakter data.** Tampung `["jml_sd", "jml_smp"]` dalam variabel *list*, tampilkan `describe()`-nya, lalu tuliskan di sel teks: berapa median masing-masing, dan apakah sebarannya lebar?
4. **Perburuan outlier.** Dengan perulangan `for` atas *list* tadi, hitung pagar IQR tiap kolom, cacah *outlier*-nya, dan berikan keterangan otomatis memakai percabangan `if-elif-else` seperti pada instruksi. Kelurahan mana pemuncak *outlier jml_sd*?
5. **Frekuensi lapangan usaha.** Rekap `value_counts()` untuk kolom kategorikal `lap_usaha_sebagian_warga` — sumber penghasilan utama sebagian besar warga tiap kelurahan. Arti kode yang muncul di data Bandar Lampung (selengkapnya lihat kamus data PODES 2021 yang dibagikan dosen):

Kode	Lapangan usaha
1	Pertanian, kehutanan, dan perikanan
3	Industri pengolahan
5	Pengadaan air, pengelolaan sampah, limbah, dan daur ulang
6	Konstruksi
7	Perdagangan besar dan eceran; reparasi mobil dan sepeda motor
8	Transportasi dan pergudangan
9	Penyediaan akomodasi dan makan minum
13	Jasa perusahaan
14	Administrasi pemerintahan, pertahanan, dan jaminan sosial wajib
17	Jasa lainnya

Tuliskan satu kalimat: lapangan usaha apa yang menjadi tumpuan terbanyak kelurahan-kelurahan Bandar Lampung, dan masuk akalkah itu untuk sebuah kota?

6. **Kelengkapan dan hubungan.** Periksa kekosongan kolom-kolom yang Anda pakai dengan `isna().sum()`, lalu hitung matriks korelasi `df[["jml_sd", "jml_smp"]].corr()`. Seberapa kuat hubungan jumlah SD dengan jumlah SMP antarkelurahan?

7. **Sintesis dan tutup.** Tutup notebook dengan satu paragraf sintesis ala laporan kompilasi: karakter sebaran fasilitas pendidikan dasar, temuan *outlier*-nya, kelengkapan datanya, dan makna korelasi SD-SMP bagi pemerataan layanan. Tutup koneksi basis data, lalu jalankan **Restart kernel & Run All** — seluruh sel harus lolos berurutan.

5 Visualisasi Data dengan Matplotlib dan Seaborn

! Sub-CPMK Pertemuan Ini

Mahasiswa mampu merumuskan visualisasi data perkotaan yang informatif menggunakan Matplotlib dan Seaborn, memilih jenis grafik yang tepat sesuai karakteristik data, menerapkan estetika grafik, dan menyusun interpretasi naratif atas visualisasi tersebut.

5.1 Pendahuluan

<masukkan narasi yang menceritakan ‘big why’ mempelajari visualisasi data dengan Matplotlib dan Seaborn di sini>

5.2 Gambaran Umum Kasus

Sebagai peserta studio perencanaan wilayah dan kota, Anda tentu akrab dengan laporan kompilasi data. Angka-angka di tabel profil kelurahan Bandar Lampung yang kita olah sebelumnya sangat sulit dibaca secara langsung oleh pengambil keputusan jika hanya disajikan dalam bentuk tabel baris-kolom yang kaku. Di sinilah visualisasi data berperan penting. Grafik yang dirancang dengan baik tidak hanya mempercepat pemahaman, tetapi juga meminimalisir kesalahan interpretasi kebijakan pembangunan kota.

Bayangkan visualisasi data sebagai proses menggambar grafik presentasi untuk sidang studio perencanaan. Jika grafik kita tidak memiliki judul yang jelas, label sumbunya terpotong, warnanya saling bertabrakan, atau legendanya hilang, asisten studio atau dosen pembimbing pasti akan langsung memberikan coretan revisi berwarna merah pada kertas gambar kita. Di modul ini, kita akan mempelajari prinsip dasar visualisasi yang baik dan mempraktikkannya untuk memvisualisasikan profil fasilitas pendidikan dan kepadatan penduduk Kota Bandar Lampung menggunakan pustaka Matplotlib dan Seaborn.

5.3 Instruksi

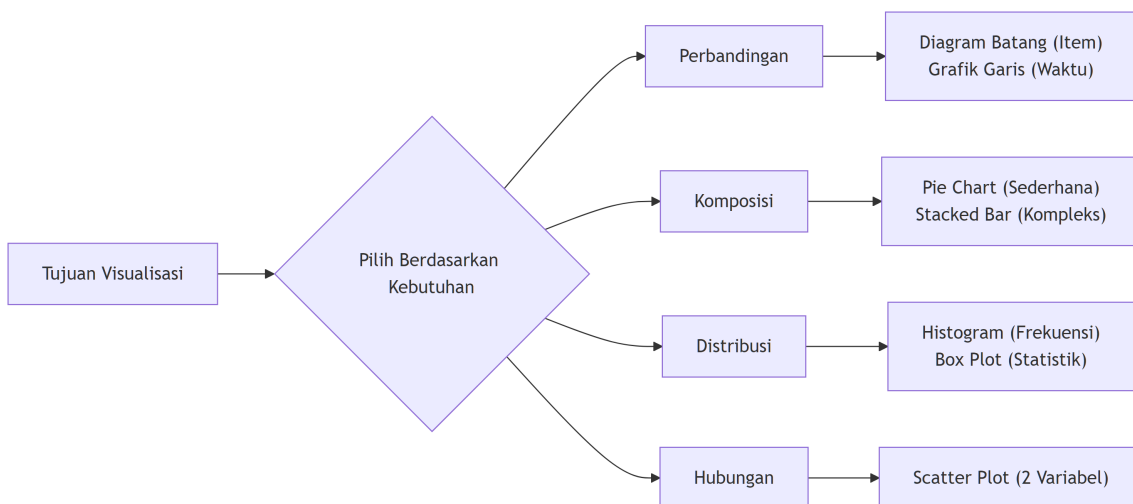
Buat notebook baru `modul/modul-05.ipynb` di direktori proyek Anda, sambungkan ke *kernel analitika-perkotaan*, dan kerjakan seluruh instruksi di dalamnya.

5.3.a Prinsip Visualisasi: Memilih Grafik Sesuai Pertanyaan Perencanaan

Sebelum mulai memprogram, pahami terlebih dahulu kerangka pikir visualisasi data yang dirancang untuk kebutuhan analisis dan presentasi laporan studio:

- **Tiga Prinsip Schwabish:**
 - *Tunjukkan Data:* Pastikan perhatian pembaca tertuju langsung pada informasi utama grafik.
 - *Kurangi Kekacauan (Reduce Clutter):* Bersihkan elemen dekoratif non-data yang mengganggu kognisi (seperti efek 3D, bayangan, atau garis kisi-kisi latar belakang yang terlalu tebal).

- **Integrasikan Teks:** Tempatkan label atau legenda sedekat mungkin dengan data untuk mengurangi beban kognitif pembaca.
- **Taksonomi Data:**
 - **Kuantitatif (Numerik):** Data yang dapat dihitung atau diukur (misalnya kepadatan penduduk atau jumlah sekolah). Data ini bisa berupa *diskrit* (bulat, seperti jumlah sekolah) atau *kontinu* (desimal, seperti luas wilayah).
 - **Kategorikal (Kualitatif):** Data pengelompokan (seperti nama kecamatan atau jenis topografi). Data ini bisa berupa *nominal* (tanpa urutan, seperti jenis lapangan usaha) atau *ordinal* (memiliki tingkatan/urutan, seperti tingkat kepuasan layanan).
 - **Analisis “So What?”:** Jangan pernah menggunakan grafik garis (*line chart*) bersambung untuk data yang bersifat diskrit antarkategori (misalnya jumlah sekolah per kelurahan). Garis yang terhubung secara implisit menunjukkan adanya nilai antara di sepanjang garis tersebut, padahal di lapangan tidak ada nilai transisi antar-kelurahan. Ini adalah kesalahan visualisasi yang fatal.
- **Pohon Keputusan Pemilihan Grafik:** Berikut adalah panduan sederhana memilih grafik yang sesuai dengan tujuan analisis Anda:



Catatan: Diagram di atas membantu kita mencocokkan pertanyaan perencanaan dengan jenis visual yang tepat secara instan.

5.3.b Persiapan Data dan Impor Pustaka

Seperti pada modul-modul sebelumnya, kita akan menggunakan basis data relasional SQLite `bd1.sqlite` yang berada di folder `dataset` sebagai sumber data utama. Impor pustaka yang diperlukan (`sqlite3`, `pandas`, `matplotlib.pyplot`, dan `seaborn`), buka koneksi basis data, lalu muat data gabungan kelurahan dan kecamatan menggunakan `LEFT JOIN` ke dalam `DataFrame` Pandas:

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Buka koneksi basis data
path_db = "../dataset-ganjil-2026-2027/bd1.sqlite"
koneksi = sqlite3.connect(path_db)
```

```
# Ambil seluruh data kelurahan beserta nama kecamatan induknya
kueri = """
SELECT
    kel.*,
    kec.nama_kec
FROM
    kelurahan kel
LEFT JOIN
    kecamatan kec ON kel.kode_kec = kec.kode_kec
"""
df = pd.read_sql(kueri, koneksi)
print(df.shape)
```

```
(126, 34)
```

5.3.c Diagram Batang: Membandingkan Fasilitas Antar-Kecamatan

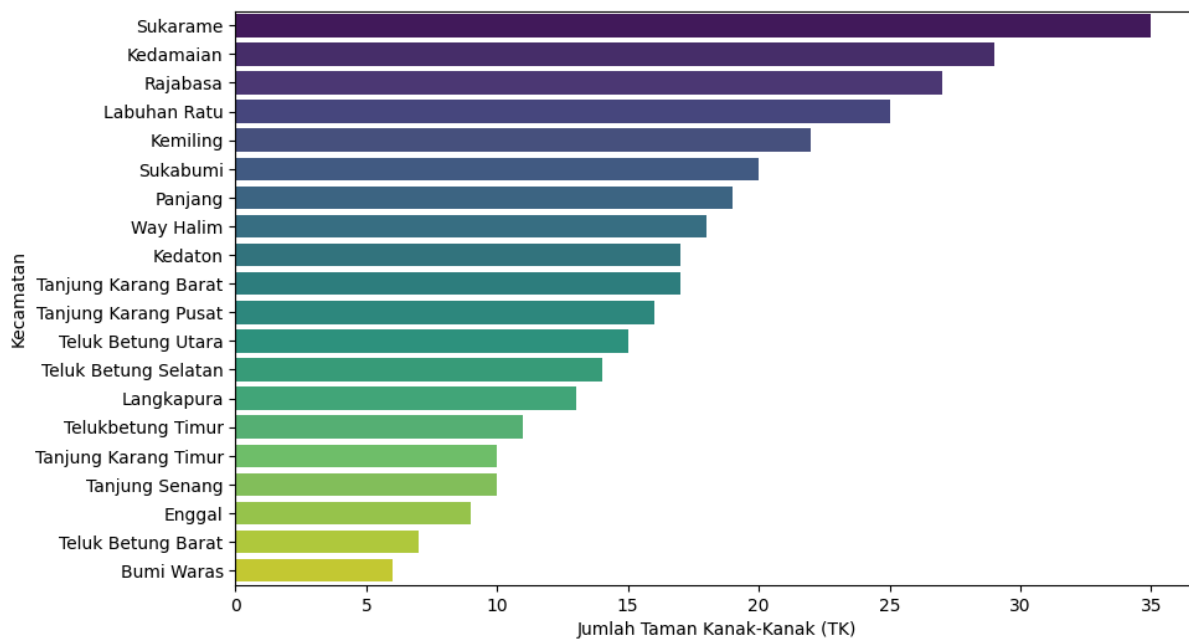
Pertanyaan pertama kita: *Kecamatan mana yang memiliki jumlah fasilitas taman kanak-kanak (TK) terbanyak?*

Untuk menjawab perbandingan kuantitas antar-item kategorikal (nama kecamatan), grafik batang adalah pilihan paling tepat. Kita perlu menghitung total TK (`jml_tk_negeri` + `jml_tk_swasta`) untuk setiap kelurahan terlebih dahulu, lalu mengelompokkannya (*group by*) berdasarkan nama kecamatan, dan mengurutkannya dari yang terbanyak.

```
# Hitung total TK di setiap kelurahan
df["total_tk"] = df["jml_tk_negeri"] + df["jml_tk_swasta"]

# Agregasikan total TK per kecamatan
tk_per_kec = df.groupby("nama_kec")["total_tk"].sum().reset_index()
tk_per_kec = tk_per_kec.sort_values(by="total_tk", ascending=False)

# Mulai membuat plot
plt.figure(figsize=(10, 6))
sns.barplot(
    data=tk_per_kec,
    x="total_tk",
    y="nama_kec",
    hue="nama_kec",
    palette="viridis",
    legend=False
)
plt.xlabel("Jumlah Taman Kanak-Kanak (TK)")
plt.ylabel("Kecamatan")
plt.show()
```



⚠ Penting!

Etika dan Batas Sumbu Y pada Grafik Batang

Grafik batang menggunakan panjang visual untuk membandingkan kuantitas. Karena itu, sumbu kuantitas (dalam diagram batang horizontal di atas adalah sumbu X, sedangkan pada diagram batang vertikal adalah sumbu Y) **wajib dimulai dari nol**. Jika Anda memotong sumbu kuantitas di atas nol, perbedaan tinggi batang akan terlihat jauh lebih dramatis daripada aslinya, sehingga membohongi pembaca secara visual. Hal ini dikategorikan sebagai distorsi data yang tidak etis.

Namun, untuk keperluan pembuktian teknis tertentu di mana Anda ingin mengatur batas sumbu secara manual (misalnya ingin menyoroti perbedaan yang sangat tipis pada grafik lain, atau memotong nilai dasar secara sengaja), Anda bisa menggunakan perintah limit pada sumbu: - Jika diagram batang vertikal atau sumbu Y: `plt.ylim(bottom=nilai_minimum)` atau `ax.set_ylim(bottom=nilai_minimum)`. - Jika diagram batang horizontal atau sumbu X: `plt.xlim(left=nilai_minimum)` atau `ax.set_xlim(left=nilai_minimum)`.

Terapkan teknik ini dengan sangat hati-hati, sertakan label nilai angka yang jelas di atas batang, dan beri catatan penjelas pada grafik Anda agar tidak menimbulkan salah tafsir.

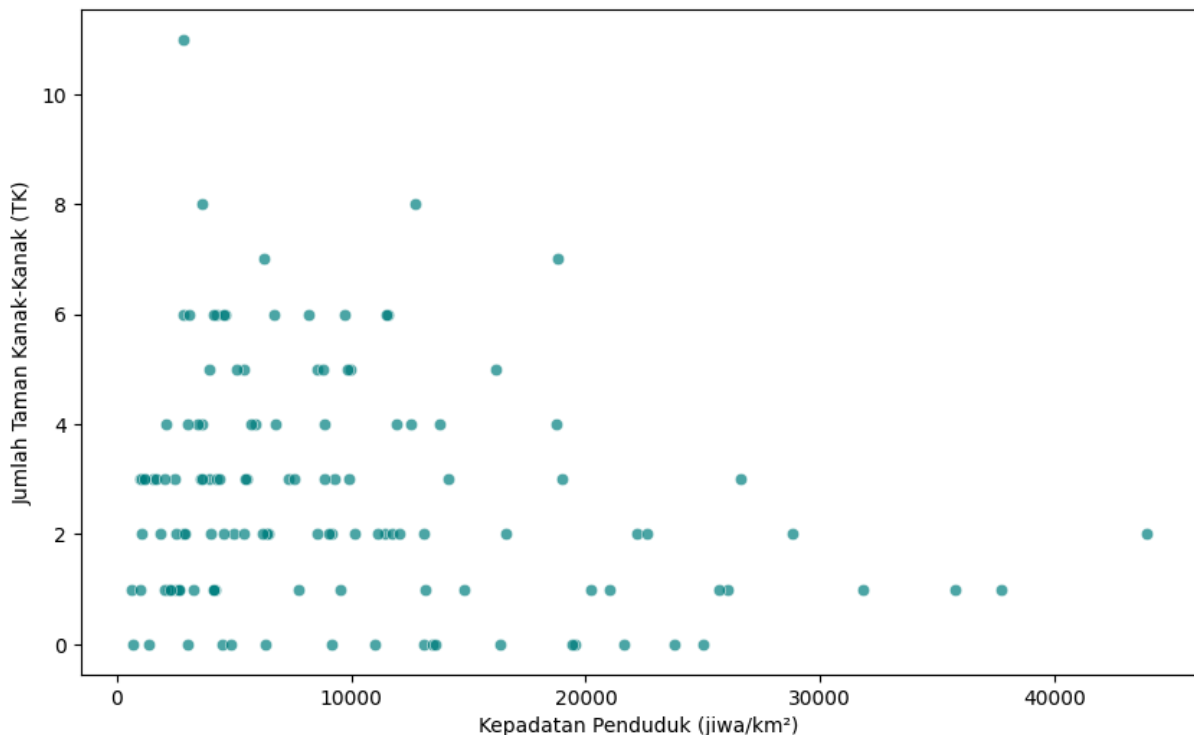
5.3.d Scatter Plot: Menelusuri Hubungan Kepadatan dan Fasilitas

Pertanyaan kedua kita: *Apakah kelurahan yang semakin padat penduduknya memiliki jumlah fasilitas TK yang lebih banyak?*

Ini adalah pertanyaan hubungan (*relationship*) antara dua variabel numerik (`kepadatan_pddk` dan `total_tk`). Sesuai pohon keputusan, grafik sebar (*scatter plot*) adalah pilihan ideal untuk melihat sebaran titik amatan ini.

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="kepadatan_pddk", y="total_tk", alpha=0.7, color="teal")
```

```
plt.xlabel("Kepadatan Penduduk (jiwa/km2)")
plt.ylabel("Jumlah Taman Kanak-Kanak (TK)")
plt.show()
```



 Pengetahuan VS Code

Fitur Interactive Plot Viewer

Ketika Anda menggunakan Visual Studio Code untuk menjalankan Jupyter Notebook, arahkan kursor mouse ke bagian grafik yang baru saja ter-render. Di sudut kanan atas gambar, VS Code secara otomatis menyediakan tombol ikon untuk mengaktifkan *Plot Viewer*. Dengan alat ini, Anda bisa memperbesar (zoom), menggeser area (*pan*), atau menyimpan file gambar secara langsung tanpa perlu mengetik kode tambahan di editor Anda.

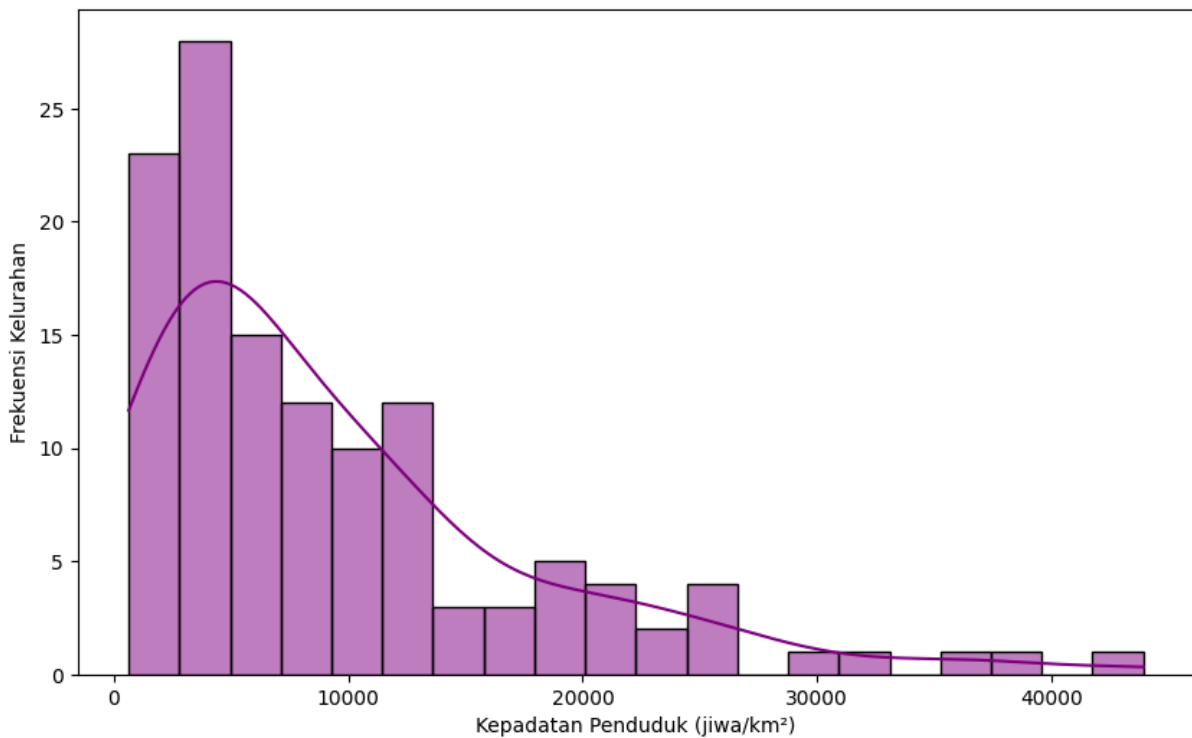
5.3.e Histogram, Density Plot, dan Box Plot: Membaca Distribusi Kepadatan Penduduk Kota

Pertanyaan ketiga kita: *Bagaimana sebaran kepadatan penduduk antarkelurahan di Bandar Lampung? Apakah mayoritas kelurahan memiliki kepadatan penduduk yang seragam, atau ada ketimpangan sebaran?*

Ini adalah pertanyaan seputar distribusi (*distribution*). Kita akan memadukan histogram (mengelompokkan nilai ke dalam tabung/bin) dengan kurva estimasi kepadatan kernel (*Kernel Density Estimate* atau KDE) untuk memvisualisasikan bentuk sebarannya:

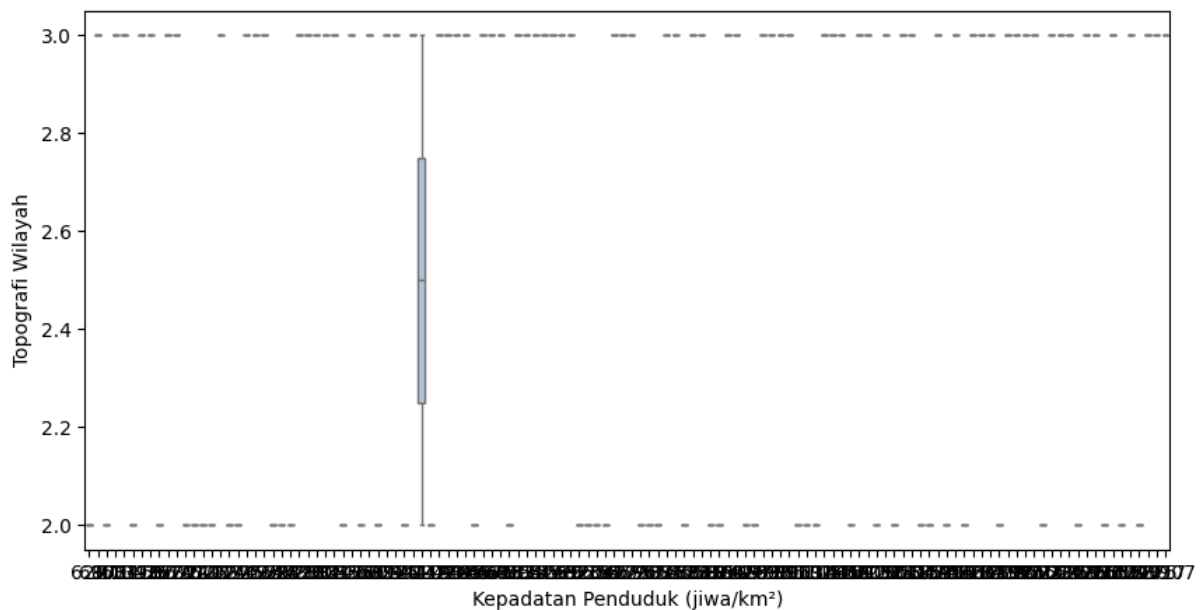
```
plt.figure(figsize=(10, 6))
sns.histplot(df["kepadatan_pddk"], kde=True, bins=20, color="purple")
```

```
plt.xlabel("Kepadatan Penduduk (jiwa/km2)")
plt.ylabel("Frekuensi Kelurahan")
plt.show()
```



Selain histogram, pohon keputusan kita juga menyebut *box plot* sebagai grafik distribusi. Jika histogram menunjukkan bentuk sebaran, *box plot* meringkas sebaran itu menjadi lima angka statistik: nilai minimum, kuartil pertama, median, kuartil ketiga, dan nilai maksimum — sementara titik-titik di luar “pagar” adalah *outlier* yang sudah kita kenali di Modul 4. Keunggulan utamanya: *box plot* mudah diijajarkan antarkelompok kategorikal. Mari bandingkan sebaran kepadatan penduduk antarjenis topografi wilayah:

```
plt.figure(figsize=(10, 5))
sns.boxplot(data=df, x="kepadatan_pddk", y="topo", color="lightsteelblue")
plt.xlabel("Kepadatan Penduduk (jiwa/km2)")
plt.ylabel("Topografi Wilayah")
plt.show()
```



5.3.f Estetika Visual: Menambahkan Judul, Label, Legenda, dan Warna Harmonis

Grafik-grafik di atas secara teknis sudah benar, tetapi tampilannya masih sangat dasar. Untuk presentasi studio yang bernilai profesional, kita wajib menerapkan estetika visual: menambahkan judul yang deskriptif, memperjelas label sumbu, merapikan legenda, dan memoles tampilannya.

Kita akan menyusun opsi pengaturan font dan warna di dalam variabel dictionary, lalu membongkarnya (*unpacking*) ke dalam perintah visualisasi untuk menjaga efisiensi penulisan kode:

```
# Mengatur gaya estetika Seaborn
sns.set_theme(style="whitegrid")

# Membuat dictionary berisi opsi teks judul agar seragam dan rapi
opsi_judul = {
    "fontsize": 14,
    "fontweight": "bold",
    "color": "darkblue",
    "pad": 15
}

opsi_label = {
    "fontsize": 11,
    "color": "dimgray"
}

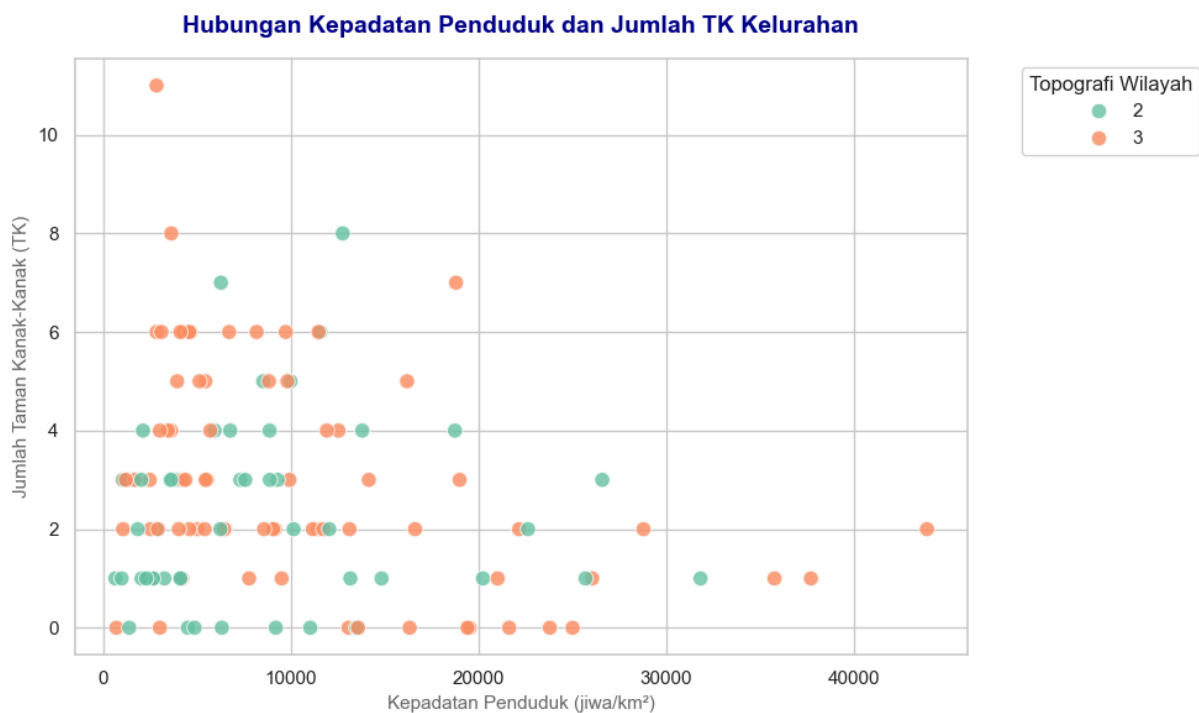
# Memulai penggambaran grafik yang sudah dipoles
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="kepadatan_pddk",
    y="total_tk",
    hue="topo",
```

```

palette="Set2",
alpha=0.8,
s=80
)

# Menerapkan dictionary opsi label ke elemen teks
plt.title("Hubungan Kepadatan Penduduk dan Jumlah TK Kelurahan", **opsi_judul)
plt.xlabel("Kepadatan Penduduk (jiwa/km²)", **opsi_label)
plt.ylabel("Jumlah Taman Kanak-Kanak (TK)", **opsi_label)
plt.legend(title="Topografi Wilayah", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```



Konsep Python

Dictionary dan Dictionary Unpacking (**)

Dictionary adalah tipe data koleksi pada Python yang menyimpan pasangan kunci (*key*) dan nilai (*value*). Berbeda dengan *list* yang diakses menggunakan indeks angka (0, 1, ...), kamus ini diakses menggunakan kunci bertipe string atau tipe data tak-berubah lainnya.

Contoh pendefinisian kamus:

```
opsi_judul = {"fontsize": 14, "color": "darkblue"}
```

Untuk menggunakan nilai di dalamnya, kita bisa memanggil kuncinya: `opsi_judul["color"]`.

Dictionary Unpacking ()** adalah operator pembongkaran. Ketika kita memanggil fungsi dan meletakkan tanda bintang ganda (**) sebelum nama variabel dictionary, Python akan secara

otomatis membongkar seluruh pasangan kunci-nilai tersebut dan meneruskannya sebagai argumen kata kunci (*keyword arguments*) ke dalam fungsi tersebut.

Jadi, penulisan:

```
plt.title("Judul", **opsi_judul)
```

Secara instan diterjemahkan oleh interpreter Python menjadi:

```
plt.title("Judul", fontsize=14, color="darkblue")
```

Teknik ini sangat berguna untuk membuat templat gaya teks yang bersih, konsisten, dan mudah diperbarui di satu baris tunggal, sehingga Anda tidak perlu mengetik argumen dekoratif berulang-ulang di setiap grafik.

i Pengetahuan Komputer

Konsep DPI (Dots Per Inch) untuk Ekspor Grafik

Saat Anda menyusun laporan studio perencanaan, gambar visualisasi tidak cukup hanya tampil di layar komputer. Grafik tersebut harus diekspor sebagai file gambar statis untuk dimasukkan ke laporan akhir (Word/PDF) atau poster presentasi.

Gunakan fungsi `plt.savefig()` untuk menyimpan grafik. Secara default, resolusi ekspor berkisar pada 72-96 DPI (*Dots Per Inch*), yang akan terlihat buram atau pecah ketika dicetak di atas kertas fisik. Untuk publikasi cetak yang tajam, biasanya mengekspor dengan resolusi minimal **300 DPI**:

```
# Menyimpan file grafik dengan ketajaman tinggi untuk cetakan
plt.savefig("hubungan_kepadatan_tk.png", dpi=300, bbox_inches="tight")
```

Argumen `bbox_inches="tight"` memastikan agar tidak ada teks label atau legenda di pinggir grafik yang terpotong saat file gambar disimpan.

5.3.g Storytelling: Menuliskan Interpretasi Naratif di Bawah Grafik

Membuat grafik hanyalah separuh tugas dari seorang analis perkotaan. Tugas terpenting berikutnya adalah menuliskan cerita di balik data (*storytelling*) dalam bentuk narasi pendek yang diletakkan tepat di bawah grafik. Narasi visual yang baik dalam studio perencanaan minimal menjawab tiga pertanyaan dasar: 1. *Pola umum apa yang terlihat secara jelas pada grafik?* 2. *Apakah ada anomali atau titik ekstrem yang menarik perhatian?* 3. *Apa rekomendasi awal atau implikasi kebijakan tata ruang dari temuan visual tersebut?*

Mari kita buat interpretasi singkat dari grafik hubungan kepadatan dan jumlah TK di atas:

Interpretasi Grafik: Grafik sebar di atas menunjukkan bahwa hubungan antara kepadatan penduduk kelurahan dan ketersediaan fasilitas TK di Kota Bandar Lampung tergolong lemah. Kelurahan-kelurahan dengan kepadatan penduduk yang sangat tinggi (di atas 20.000 jiwa/km²) ternyata tidak otomatis memiliki fasilitas TK yang melimpah; sebagian besar menumpuk di bawah

5 unit TK. Hal ini mengindikasikan adanya isu kesenjangan layanan pendidikan anak usia dini. Selain itu, sebaran warna topografi memperlihatkan bahwa wilayah dataran memiliki variasi fasilitas yang lebih luas dibandingkan dengan wilayah lembah/aliran sungai yang cenderung terbatas. Pemerintah Kota disarankan memprioritaskan bantuan penyediaan ruang pendidikan di wilayah-wilayah padat penduduk yang saat ini minim layanan TK.

Tutup sesi kerja basis data Anda:

```
# Tutup koneksi basis data
koneksi.close()
```

5.4 Latihan Mandiri

Kerjakan latihan ini di berkas notebook baru bernama `modul/latihan-05.ipynb` menggunakan *kernel* `analitika-perkotaan`. Pastikan alur kode Anda rapi dan terstruktur, serta jalankan perintah **Restart kernel & Run All** untuk memverifikasi bahwa kode Anda bebas dari galat.

Latihan ini menggunakan dataset kelurahan yang sama, namun dengan fokus analisis fasilitas pendidikan dasar (**SD dan SMP**) serta struktur lapangan usaha warga:

1. **Persiapan Data:** Hubungkan notebook ke basis data `bd1.sqlite` dan tarik seluruh data kelurahan beserta kecamatan induknya ke DataFrame menggunakan Pandas.
2. **Variabel Baru:** Buat dua kolom baru untuk menghitung total fasilitas pendidikan dasar:
 - `total_sd = jml_sd_negeri + jml_sd_swasta`
 - `total_smp = jml_smp_negeri + jml_smp_swasta`
3. **Diagram Batang Data Kategorikal:** Hitung frekuensi tiap kategori lapangan usaha utama warga (`lap_usaha_sebagian_warga`) menggunakan `value_counts()` yang sudah Anda kenal di Modul 4, lalu sajikan hasilnya sebagai diagram batang. Urutkan batang dari kategori dengan frekuensi tertinggi.
4. **Scatter Plot:** Buat sebuah grafik sebar (*scatter plot*) untuk melihat hubungan antara jumlah SD (`total_sd`, sumbu X) dengan jumlah SMP (`total_smp`, sumbu Y) per kelurahan.
5. **Box Plot Distribusi:** Visualisasikan distribusi `total_sd` antar-kelurahan pada setiap kecamatan menggunakan *box plot* (`sns.boxplot` dengan sumbu Y berisi `nama_kec`). Amati kecamatan mana yang sebarannya paling lebar dan mana yang memiliki *outlier*.
6. **Estetika Diagram:** Poles diagram batang pada langkah 3 menggunakan opsi parameter kamus/dictionary: terapkan judul yang informatif, label sumbu yang bersih, warna palet yang harmonis, dan simpan grafik tersebut menggunakan `plt.savefig()` dengan resolusi 300 DPI.
7. **Interpretasi & Tutup:** Tuliskan satu paragraf interpretasi singkat (*storytelling*) mengenai kesimpulan visualisasi Anda terhadap ketersediaan fasilitas SD dan SMP serta struktur lapangan usaha warga, lalu tutup koneksi basis data Anda.